Multi-Asset Next-Day Price Prediction System
Bitcoin · Gold · Silver

Complete Technical Report of the Final Year Project — prepared for supervisor review

Author: Alyan Shahid (teamlocalhost) · **Programme:** BS Computer Science, University of Lahore
Report date: 13 November 2026 · **Code revision documented:** git commit c854397 + the uncommitted changes of the 13 November 2026 audit (branch main)

Basis of this document. Every statement about data, features, models, metrics and behaviour was taken from the implementation itself — the Python sources under `src/`, `app/` and `config.py`, the frozen datasets under `data/`, the trained artefacts under `data/models/` and the result files under `results/`. All tables are generated programmatically from those files by `docs/build_technical_report.py`; nothing is typed in by hand. The automated test suite (55 tests) and a live prediction run were executed on the report date to confirm the system works as described.

Scope note. Where the project explored something that is *not* part of the final served system (for example the stacked-ensemble experiment, the multi-day and volatility targets, or the automatic daily prediction record that was prototyped and then removed), this report says so explicitly rather than describing it as a feature.

Edition of 13 November 2026. This edition replaces the report of 14 September 2026 and documents the final code. Changes since then: two moving-average baselines (MA-5, Mean-20d); the ≈ 6-year pooled walk-forward evidence (§14.5); the **calibrated** 68 % uncertainty band $P \cdot \exp(\hat{r} \pm k \cdot \sigma_t)$ that replaced the earlier ±1-RMSE band (§16.2); a live scorecard on the Forecast tab and a Predict-a-Day list that grows with new live days (§23); the fresh hold-out on days published after the data were frozen (§22); and the audit of 13 November 2026 (§32), which restored a raw data file that an editor had re-formatted, repaired the CI workflow and pinned the environment. Every table is regenerated from the result files.

Contents

Contents . . . 2
1. Executive Summary . . . 4
2. Project Overview — the complete pipeline . . . 5
3. Project Objectives . . . 6
4. Problem Statement . . . 6
5. Assets Predicted . . . 7
6. Dataset — Detailed Explanation . . . 7
6.1 Columns of the raw files . . . 8
6.2 Data quality of the stored files . . . 8
7. Data Preprocessing . . . 9
8. Feature Engineering — Complete List . . . 10
9. Target Variable . . . 11
10. Time-Series Data Splitting . . . 12
11. Data Leakage Prevention . . . 12
12. Machine-Learning Models . . . 14
12.1 Why these models and what each contributes . . . 14
13. Model Training . . . 15
14. Model Performance . . . 17
14.1 Unseen test set — every model, every asset . . . 17
14.2 Served forecast vs random walk — summary . . . 18
14.3 Walk-forward validation (model-selection evidence) . . . 19
14.4 Performance across market regimes (test period) . . . 20
14.5 Long-run evidence — ≈ 6 years of pooled walk-forward predictions . . . 20
15. Baseline Comparison . . . 21
15.1 Before / after comparison on identical unseen days . . . 22
15.2 The archived "62 % Silver" result . . . 22
16. Ensemble / Combined Prediction — the served forecast . . . 23
16.1 The stacked ensemble (experiment, not served) . . . 23
16.2 The calibrated 68 % uncertainty band . . . 23
17. How a User Generates a Prediction . . . 24
17.1 Example run (actual output on 13 November 2026 — illustrative only, not a result) . . . 24
18. UP / DOWN Prediction . . . 25
19. Automatic Daily Prediction System — status in the final implementation . . . 25
20. HIT / FAIL Methodology . . . 26
21. Prediction History (per-day test predictions) . . . 26
22. Real-World Forward Testing — what the project can and cannot claim . . . 27
23. Streamlit Application . . . 28

24. Visualisations 35
25. Technology Stack 36
26. Project Architecture 36
27. File / Code Structure 38
28. Error Handling & Reliability 39
29. Limitations 39
30. Ethical / Academic Positioning 40
31. Future Improvements 40
32. Final Project Status — honest assessment 41
33. Final Technical Summary 42

1. Executive Summary

What the project does. The system predicts the **next trading day's closing price** of three assets — Bitcoin (BTC-USD), Gold (COMEX front-month futures, GC=F) and Silver (SI=F) — from daily market data downloaded from Yahoo Finance. Rather than predicting the price directly, every model predicts the next-day **log return** $\ln(P_{t+1}/P_t)$ from 28–29 backward-looking, stationary features, and the predicted return is converted back into a dollar price. The direction of the predicted return gives the **UP / DOWN** call.

Why these three assets. They span three market structures with different behaviour: a 24/7 crypto-asset with very high volatility (Bitcoin, daily σ ≈ 3.4 %), a comparatively calm exchange-traded safe-haven commodity (Gold, σ ≈ 1.0 %) and a more volatile industrial/precious metal that co-moves with gold (Silver, σ ≈ 1.9 %). They are liquid, have long free daily histories, and are influenced by macro variables (US dollar, yields, equities, volatility) that can be added as features.

Machine-learning approach. Six models are trained per asset — Ridge regression, Random Forest, LightGBM, CatBoost, a GRU and an LSTM network — and compared against five baselines (random walk, historical mean, ARIMA and two moving-average rules, MA-5 and Mean-20d) and a stacked-ensemble experiment. Hyper-parameters are chosen with 4-fold expanding-window walk-forward validation on the training/validation period only; the untouched test period (2026-02-18 → 2026-09-12) is evaluated exactly once. **The served forecast is the equal-weight combination** of the six trained models' predicted returns: the user never selects a model.

What the user sees. A Streamlit dashboard with four tabs: *Forecast* (one click → the latest complete market bar is fetched, all six models run, one final predicted price with UP/DOWN, the change in % and $, a **calibrated 68 % uncertainty band**, each model's own prediction, a **live scorecard** that re-runs every model on the most recent 20–90 days and scores it against the actual closes, and the held-out reliability of the forecast), *Predict a Day (unseen test)* (stand on any day after the training data, predict the next day using only data up to that day, then reveal the actual close, the error and HIT/FAIL; newly published days are added automatically), *Model Performance* and *Methodology*. A FastAPI endpoint serves the same forecast as JSON.

How predictions are evaluated. In return space (RMSE, MAE, R^2), in dollar space (MAE, RMSE, MAPE), directional accuracy on non-flat days with a binomial p-value against 50 %, a Diebold–Mariano test against the random-walk forecast, and a long/flat strategy back-test with 10 bps costs. The random walk ("tomorrow = today") is the baseline every model must beat.

Headline result (honest). On the unseen test period the combined forecast's next-day price error is 1.52 % (Bitcoin), 1.29 % (Gold) and 2.47 % (Silver) of price — essentially identical to the random walk (1.52 / 1.29 / 2.49 %). Directional accuracy is 46.9 / 50.3 / 53.8 % (binomial p = 0.83 / 0.50 / 0.20) and the Diebold–Mariano test cannot reject equal accuracy with the random walk (p = 0.49 / 0.97 / 0.46). **No model, single or combined, beats the random walk at a one-day horizon in this data.** This is the expected result for liquid daily financial returns and the project reports it as such; the project's contribution is a leakage-audited, statistically tested pipeline and a transparent demonstration system, including the discovery and correction of a subtle close-time data leak that had produced an apparently significant 62 % directional-accuracy result for Silver. The same answer holds on ≈ 6 years of pooled walk-forward predictions (§14.5) and on the days published after the data were frozen (§22).

Automatic daily predictions — status. An automatic daily forward-prediction record (scheduled job that saves each day's forecast and verifies it against the next day's actual close with HIT/FAIL) was built and **removed from the final system at the author's decision** (14 September 2026). The final code base contains *no scheduler and no persistent forward log*. Out-of-sample evidence comes instead from the unseen test period (every test day predicted from data up to that day), the live scorecard (the most recent days, re-computed on demand) and the fresh hold-out on the days published after the data were frozen (§22). Section 19 explains this in detail.

Why it is suitable as an FYP. The project covers the complete data-science life-cycle — data acquisition from public APIs, cleaning, feature engineering, leakage control, time-series validation, hyper-parameter tuning, twelve forecasting methods (six trained models, five baselines, a stacked ensemble), statistical evaluation, model combination, a calibrated uncertainty band, a user-facing dashboard, an API, 55 automated tests run by a CI workflow, a pinned environment and reproducible documentation — and it demonstrates scientific honesty: results were re-run and corrected when the audit found problems, and negative results are reported as results.

2. Project Overview — the complete pipeline

The system is organised as a sequence of scripts, each of which reads the previous stage's files and writes its own. The `Makefile` target `make pipeline` runs them in order; `scripts/retrain.py` additionally re-downloads the data. Figure 1 shows the stages actually implemented.

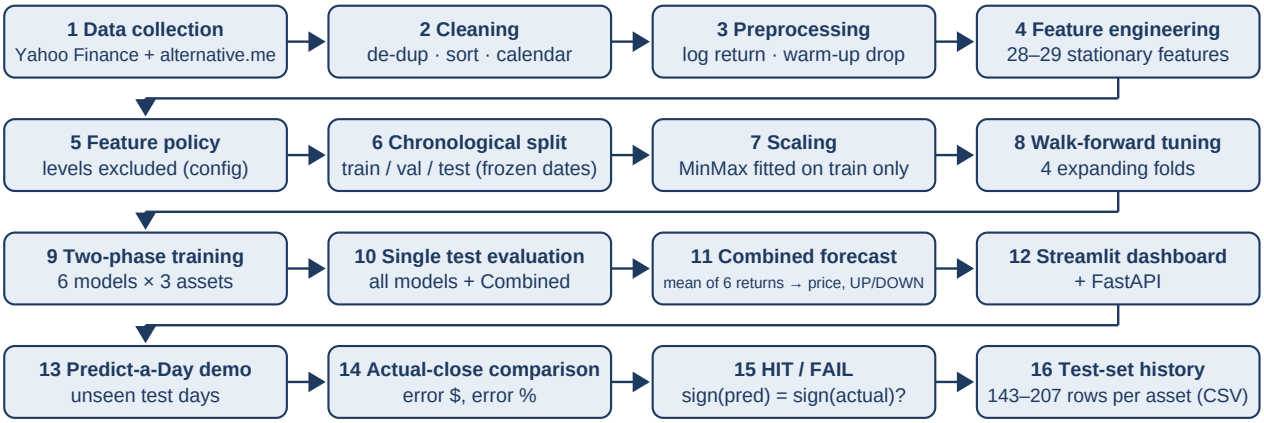


Figure 1. The implemented pipeline. Stages 1–7 live in `src/data/`*, 8–9 in* `src/training/`*, 10 and 16 in* `src/evaluation/backtesting.py`*, 11 and 13–15 in* `src/inference/prediction.py`*, 12 in* `app/streamlit_app.py` *and* `src/api/app.py`*.*

Stage	What happens	Implemented in
Data collection	Daily OHLCV for BTC-USD, GC=F, SI=F and daily closes for DXY, WTI, 10-y yield, S&P 500, VIX from Yahoo Finance (yfinance); Bitcoin Fear & Greed index from alternative.me. The running (incomplete) bar of the current day is dropped at download time (complete-bar rule).	src/data/data_collection.py, external_data.py, market_calendar.py
Data cleaning	Drop duplicate timestamps, sort chronologically, drop non-positive prices; Bitcoin re-indexed to every calendar day (gaps forward-filled — none in the stored data); Gold/Silver keep exchange days only (no synthetic weekend rows).	preprocessing.py :: DataCleaner.clean_data
Preprocessing	Log return $\ln(P_t/P_{t-1})$; indicator warm-up rows dropped (dropna); validation: no NaN/inf, ≥ 100 rows.	preprocessing.py
Feature engineering	Returns and lags, momentum, rolling/HAR/EWMA volatility, RSI, ADX, ROC, normalised MACD, EMA ratios, Bollinger %B/width, ATR/price, high–low range, macro returns, sentiment/calendar/volume (Bitcoin), gold return (Silver). Close-time rule for the metals.	preprocessing.py (add_* methods)
Feature selection	A fixed policy rather than an algorithm: all raw price-level columns are excluded from model inputs (config.LEVEL_COLUMNS); ablation experiment E2 confirmed no feature group hurts.	config.py, src/experiments/run_experiments.py
Train / validation / test	Chronological split by frozen calendar dates: train ≤ 2025-09-10, validation ≤ 2026-02-17, test = remainder; defined by the dates the target covers.	preprocessing.py :: normalize_and_split, build_dataset
Model training	Two-phase fit: phase A on train with validation monitoring (stopping point; validation metrics mildly optimistic for the early-stopped models), phase B refit on train+val with the stopping point fixed → deployed artefact.	src/training/train_models.py, src/models/registry.py
Model evaluation	Walk-forward CV table (selection evidence) and ONE evaluation of every model on the untouched test set with return-space, USD, directional, Diebold–Mariano and strategy metrics.	src/evaluation/backtesting.py, src/utils/metrics.py
Combined prediction	Equal-weight mean of the six trained models' predicted log returns → price via $P_t \cdot \exp(\hat{y})$ → UP if $\hat{y} > 0$ else DOWN.	src/inference/prediction.py
Dashboard / API	Streamlit dashboard (4 tabs) and FastAPI (/health, /models, /predict/{asset}).	app/streamlit_app.py, src/api/app.py
Verification & history	Predict-a-Day (unseen test day → predict → reveal actual → error → HIT/FAIL) , the per-day prediction files (one row per unseen day, all models + Combined) written by the evaluation script, and the live scorecard.	prediction.py :: predict_for_date, prediction_history, recent_track_record; results/predictions/*.csv

Table 1. Pipeline stages and where they are implemented.

3. Project Objectives

The objectives below are the ones the delivered code actually meets; each is paired with the evidence in the repository.

#	Objective	How it is met (evidence)
1	Collect historical daily financial data for three assets plus macro/sentiment drivers from free public sources	Yahoo Finance + alternative.me downloads from 2018-01-01; 3,177 / 2,186 / 2,186 raw rows (BTC / Gold / Silver)
2	Process financial time series correctly (calendar handling, complete bars only, no look-ahead)	DataCleaner; complete-bar rule; close-time alignment; tests in test_features.py and test_market_calendar.py
3	Engineer meaningful, stationary, backward-looking predictive features	29 / 28 / 29 model features (Table 8); ADF tests in results/eda_summary.csv
4	Predict the next trading day's price (via the log return) with a one-day horizon	build_dataset(task="return_1d"); reconstruct_price
5	Train and compare several model families under a valid time-series protocol	6 trained models + 5 baselines + stacked experiment; walk-forward CV; two-phase training
6	Compare every model against a meaningful baseline with statistical tests	Random-walk baseline; Diebold–Mariano test; binomial test on direction; pooled walk-forward tests over ≈ 6 years (§14.5); fresh hold-out (§22)
7	Combine the models into one served forecast that needs no manual model choice	COMBINED = equal-weight mean; evaluated like a single model (row "Combined")
8	Visualise data, predictions and evaluation for a non-expert user	Streamlit dashboard, Plotly charts, matplotlib report figures
9	Evaluate predictions against actual values on genuinely unseen days, per day	results/predictions/*.csv (predicted / actual / error / HIT-FAIL per unseen day); Predict-a-Day; live scorecard; fresh hold-out
10	Provide an easy-to-use dashboard and an API	One "Run prediction" button; FastAPI /predict/{asset}
11	Make the work reproducible and testable	Frozen split dates, seeds, pinned lock file, Makefile, 55 tests, CI workflow (preprocessing + tests on every push)

Table 2. Objectives and evidence.

Not an objective of the final system: a persistent automatic daily prediction log (prototyped, then removed — §19), multi-day or volatility forecasting (evaluated as experiments E3/E4 only), or any trading recommendation.

4. Problem Statement

Short-horizon prediction of financial asset prices is one of the hardest forecasting problems, for reasons that are visible in this project's own data:

- **Volatility and fat tails.** Daily returns are large and heavy-tailed: excess kurtosis is 15.2 (Bitcoin), 3.6 (Gold) and 5.2 (Silver) in the training window; 39 % of Bitcoin days move more than 2 %.
- **Almost no linear memory.** The lag-1 autocorrelation of returns is -0.053 / -0.027 / -0.021: yesterday's return says almost nothing about today's, which is why ARIMA-type models can add little.
- **Non-stationarity and regime change.** Price levels are non-stationary (ADF p ≈ 0.97–1.00) while returns are stationary ($p < 10^{-29}$); the 2026 test period is a different regime — Gold traded at $3,986–$5,294 against a training range of $1,176–$3,644.
- **Non-linear, unstable relationships and external shocks.** Macro news, regulation, exchange outages and sentiment move prices in ways that are not in the history.
- **Time-series dependencies.** Observations are ordered; random shuffling, k-fold cross-validation or features that peek even a few hours into the future silently inflate results.

How the project approaches it. The problem is framed as a supervised regression of the next-day log return on information that is provably available when the day-*t* close is fixed. The random walk (predicted return = 0) is treated as the null hypothesis; models are considered useful only if they beat it under a walk-forward protocol and a single held-out evaluation with formal tests. The system is positioned strictly as an **academic prediction and evaluation system**: it does not claim, and its own results do not show, a profitable trading edge.

5. Assets Predicted

All three assets share the same target definition, split dates, model set and combination rule. They differ in data source ticker, trading calendar, close time and the feature groups that make sense for them.

	Bitcoin	Gold	Silver
Ticker / instrument	BTC-USD (Yahoo composite spot index)	GC=F (COMEX front-month gold futures)	SI=F (COMEX front-month silver futures)
Trading calendar	7 days a week (calendar days)	Exchange days only (Mon–Fri, no holidays modelled)	Exchange days only
Daily close used	00:00 UTC bar close (after every US market close)	13:30 ET COMEX settlement (Yahoo's daily Close)	13:25–13:30 ET COMEX settlement
What is predicted	Next calendar day's close (via log return)	Next exchange day's settlement (via log return)	Next exchange day's settlement (via log return)
Horizon	1 day	1 trading day	1 trading day
Usable rows (after warm-up)	3,147 (2018-01-31 → 2026-09-12)	2,156 (2018-02-15 → 2026-09-11)	2,156 (2018-02-15 → 2026-09-11)
Model features	29: price/technical set + S&P 500 and VIX same-day returns + Fear & Greed + day-of-week + volume change	28: price/technical set + previous-day DXY, WTI, 10-y yield, S&P 500, VIX returns	29: as Gold + gold's same-day return
Close-time rule	Same-day macro values allowed (bar closes after them)	Macro = previous day; hl_range / ATR / ADX lagged one session	Same as Gold
Models	6 trained (Ridge, RF, LightGBM, CatBoost, GRU, LSTM) + baselines	same	same
Served forecast	Combined (equal-weight mean of 6)	Combined	Combined
Best single model by walk-forward CV (for comparison)	CatBoost	CatBoost	GRU
Train / val / test samples	2,750 / 160 / 207	1,874 / 109 / 143	1,874 / 109 / 143

Table 3. The three assets side by side.

Important differences. (i) Bitcoin has ~45 % more rows per year because it trades on weekends; the metals keep their exchange calendar and the project deliberately does *not* create synthetic weekend rows for them (an earlier version did, and 30 % of its metal rows were forward-filled zero-return fakes). (ii) Yahoo's daily close for the futures is the 13:30 ET settlement, which is *before* the US equity, rates and FX closes — so the metals may only use the *previous* day's macro values and the previous session's High/Low (§11). (iii) The yfinance volume column for futures is front-month contract volume that collapses at contract rolls; it is not used for the metals. (iv) Yahoo publishes Bitcoin's previous-day bar several hours after 00:00 UTC, so a morning "Run prediction" may start from the bar before; the dashboard says so explicitly.

6. Dataset — Detailed Explanation

All series are downloaded by the project's own scripts and stored as CSV files under `data/raw/` (frozen evaluation data) and `data/raw_live/` (refreshed copies used only for live forecasts). No paid data, no API key and no database are used. The figures below are read from the stored files.

Asset	Data source / ticker	Date range (stored)	Frequency	Raw rows	Usable rows	Target
Bitcoin	Yahoo Finance (yfinance) · BTC-USD	2018-01-01 → 2026-09-12	daily (calendar days)	3,177	3,147	next-day log return $\ln(P_{t+1}/P_t)$
Gold	Yahoo Finance (yfinance) · GC=F	2018-01-02 → 2026-09-11	daily (exchange days)	2,186	2,156	next-day log return $\ln(P_{t+1}/P_t)$
Silver	Yahoo Finance (yfinance) · SI=F	2018-01-02 → 2026-09-11	daily (exchange days)	2,186	2,156	next-day log return $\ln(P_{t+1}/P_t)$

Table 4. Primary (target) datasets. "Usable rows" = after the 30-row indicator warm-up is dropped.

Series	Source / ticker	Rows	Date range	Used by	Feature derived
US Dollar Index	Yahoo · DX-Y.NYB	2,187	2018-01-02 → 2026-09-11	Gold, Silver	dxy_return (previous day)
WTI crude oil	Yahoo · CL=F	2,187	2018-01-02 → 2026-09-11	Gold, Silver	oil_return (previous day)
10-year Treasury yield	Yahoo · ^TNX	2,185	2018-01-02 → 2026-09-11	Gold, Silver	tnx_return (previous day)
S&P 500	Yahoo · ^GSPC	2,185	2018-01-02 → 2026-09-11	all	sp500_return (same day BTC / previous day metals)
VIX	Yahoo · ^VIX	2,187	2018-01-02 → 2026-09-11	all	vix_return (same day BTC / previous day metals)
Crypto Fear & Greed index	alternative.me API (free, no key)	3,143	2018-02-01 → 2026-09-13	Bitcoin	fear_greed (0–100, same day)
Gold close (for Silver)	the gold_data.csv above	2,186	—	Silver	gold_return (same day; same settlement window)

Table 5. External (explanatory) series.

6.1 Columns of the raw files

Column	Type	Meaning	How it is used
timestamp	datetime (tz-naive, daily)	Trading day of the bar	Index; chronological sorting and splitting; date arithmetic for the target
open	float	First traded price of the day	Chart only (level column, never a model input)
high	float	Highest price of the session	Inputs to ATR, ADX, hl_range (lagged one session for the metals)
low	float	Lowest price of the session	as high
price	float	Yahoo "Close" — BTC: 00:00 UTC close; futures: 13:30 ET settlement	Basis of the log return (target and features); P_t for price reconstruction; chart
volume	int	Traded volume (BTC: composite; futures: front-month contract volume)	Bitcoin only: log_volume_change. Not used for the metals (contract-roll jumps)
price (external files)	float	Daily close of the macro series	Converted to a daily log return on its own calendar, then aligned
fear_greed	int 0–100	Sentiment index (0 = extreme fear, 100 = extreme greed)	Bitcoin feature, forward-filled onto the BTC calendar

Table 6. Column dictionary of the stored raw files (verified against data/raw/.csv).*

6.2 Data quality of the stored files

The stored raw files contain **no null values and no duplicate timestamps** (checked on the report date). Bitcoin has a complete calendar (3,177 rows for 3,177 calendar days — no gaps to forward-fill). Gold and Silver have 2,186 exchange days each. Days with an exactly-zero return exist only in the metals (9 Gold, 3 Silver over the whole history, none inside the test period); they are excluded from directional accuracy because their direction is undefined.

Known vendor issues that the code handles. Yahoo returns the current day's *running* bar as if it were complete — the complete-bar rule (`src/data/market_calendar.py`) drops it at download time (Bitcoin: rows dated today UTC; futures and US series: rows dated today ET before 17:15 ET). Yahoo's continuous futures series follows the current front-month contract, so recent closes can differ between two downloads after a roll; the frozen evaluation files are therefore never overwritten by live refreshes.

Known vendor issue that is documented, not corrected. The free continuous futures series contains degenerate bars with High = Low: 130 Gold days and 384 Silver days (18 %), mostly 2018–2021, many with zero volume. On those days the range-based features (hl_range, atr_norm, ADX) are flattened. The bars were kept because the close is valid; feature ablation E2 shows that these features carry no predictive value either way, so the effect on the results is negligible.

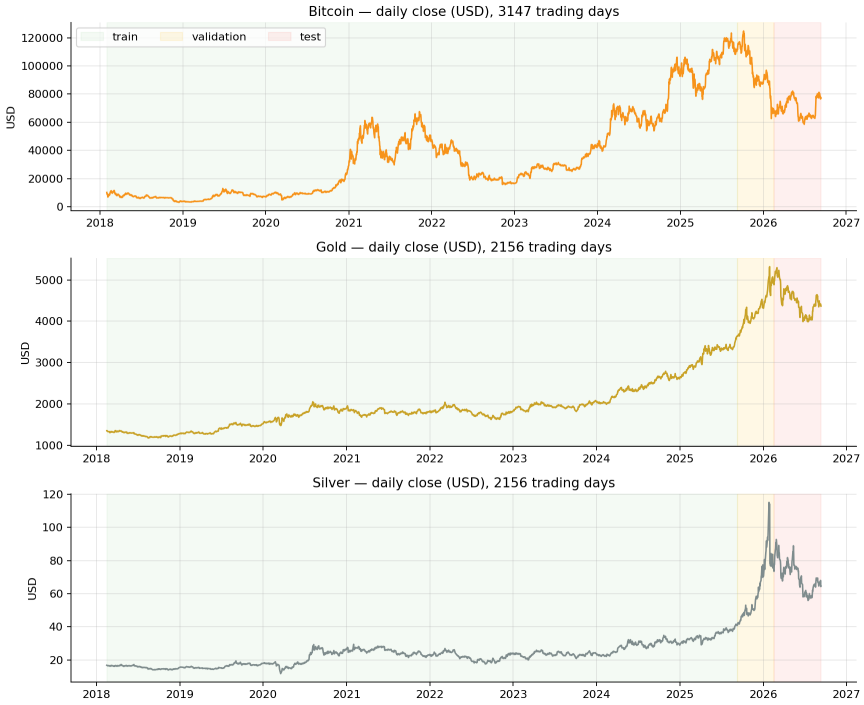


Figure 2. Stored price history of the three assets with the frozen train (green) / validation (amber) / test (red) windows.

7. Data Preprocessing

Preprocessing is implemented in `DataCleaner.clean_data()` and `normalize_and_split()` (`src/data/preprocessing.py`) and runs identically for the frozen data and for live refreshes.

Step	What is done	Why it is necessary
1 Parse & sort	timestamp parsed to datetime; rows sorted chronologically	Every later step (returns, rolling windows, split) assumes time order
2 Duplicate removal	drop_duplicates(subset=["timestamp"])	A duplicated day would double-count a sample and break the shift/rolling logic
3 Invalid values	rows with price ≤ 0 dropped; rows with missing close dropped at download	log(price) is undefined for non-positive prices
4 Calendar handling	Bitcoin: reindex to every calendar day and forward-fill genuine gaps (0 in stored data). Gold/Silver: keep exchange days only	Bitcoin trades daily, a missing day is a data gap; futures do not trade at weekends, so synthetic rows would create fake zero-return samples
5 Log return	log_return = $\ln(\text{price}_t / \text{price}_{t-1})$	Stationary quantity; basis of the target and of most features
6 Feature engineering	technical, volatility, momentum, macro, sentiment features (§8)	Turn the raw series into scale-free predictors
7 Close-time lagging	metals only: hl_range, atr_norm, ADX shifted one session; macro series aligned to the previous day	Yahoo's futures close is the 13:30 ET settlement; later information would leak (§11)
8 Warm-up removal	dropna() removes the first ~30 rows where indicators are undefined	Rolling/EWM windows need history; NaNs cannot enter a model
9 Validation	assert: no NaN, no inf, ≥ 100 rows (raises ValueError otherwise)	A corrupt download must fail loudly instead of producing silent nonsense
10 Chronological split	train ≤ 2025-09-10; validation ≤ 2026-02-17; test = later rows	Time-series data must be split forward in time (§10)
11 Scaling	MinMaxScaler(0–1) fitted on the training rows only and applied to validation/test; saved as _scaler.pkl	The recurrent models need bounded inputs; fitting on train only prevents test statistics from leaking
12 Target standardisation	target z-scored with the training mean/std (stored in model_status.json)	GRU/LSTM train on a unit-variance target; tree models are unaffected; inverted at prediction time
13 Feature alignment	the last row of each 30-day window feeds the tabular models; the whole window feeds GRU/LSTM; both see identical samples	Every model is compared on exactly the same days

Table 7. Preprocessing steps (in execution order).

Outputs per asset: `data/processed/<prefix>_features.csv` (all columns incl. price levels, for charts and for inference), `<prefix>_train_scaled.csv / _val_scaled.csv / _test_scaled.csv` (model feature columns only, scaled) and `data/models/<prefix>_scaler.pkl`.

8. Feature Engineering — Complete List

The list below is the exact set of model inputs stored in `model_status.json → features` and in the scaled split files (29 for Bitcoin, 28 for Gold, 29 for Silver). Every feature at row *t* uses only information available when P_t is observed ("historical/current only" — verified by the look-ahead tests). No price-level column is a model input.

Feature	Category	What it represents / how it is calculated	Why it may help	Assets
log_return	Return	$\ln(P_t/P_{t-1})$ — today's return	Short-term reversal / continuation	all
return_1d, return_2d, return_5d, return_10d	Lag	log_return shifted by 1, 2, 5, 10 rows (strictly past returns)	Lets tabular models see the recent path	all
return_20d	Momentum	Σ of the last 20 log returns (one-month momentum)	Momentum / monthly reversal literature	all
volatility_10d, volatility_30d	Volatility	rolling standard deviation of log_return over 10 / 30 rows	Volatility clustering	all
rv_1d, rv_5d, rv_22d	Volatility (HAR)	√(mean of r² over 1 / 5 / 22 rows) — daily, weekly, monthly realised volatility	HAR structure of volatility	all
ewma_vol	Volatility	RiskMetrics EWMA σ, λ = 0.94 (min 22 rows)	Fast-reacting volatility state	all
RSI	Momentum indicator	Wilder RSI(14) on price, 0–100	Overbought / oversold	all
ADX †	Trend indicator	ADX(14) from high/low/close (ta library); metals: previous session	Trend strength / regime	all
ROC	Momentum indicator	$(P_t/P_{t-12} - 1)\cdot 100$	12-day rate of change	all
ema14_ratio, ema30_ratio	Trend (relative)	$P_t/EMA_{14} - 1$, $P_t/EMA_{30} - 1$	Distance from short / medium trend, scale-free	all
macd_norm	Trend (relative)	$(EMA_{12} - EMA_{26}) / P_t$	MACD line normalised by price	all
macd_hist_norm	Trend (relative)	$(MACD - Signal_9) / P_t$	MACD histogram normalised	all
bb_pctb	Volatility band	$(P_t - BB_{lower}) / (BB_{upper} - BB_{lower})$, 20-day ±2σ bands	Position inside the band	all
bb_width	Volatility band	$(BB_{upper} - BB_{lower}) / BB_{mid}$	Band width = volatility regime	all
atr_norm †	Volatility	ATR(14) / P_t; metals: previous session	True range relative to price	all
hl_range †	Volatility (range)	(High − Low) / P; metals: previous session's range	Parkinson-style intraday volatility proxy	all
sp500_return ‡	Macro	$\ln(SPX_t/SPX_{t-1})$ — same day for BTC, previous day for metals	Risk-on / risk-off	all
vix_return ‡	Macro	$\ln(VIX_t/VIX_{t-1})$ — same day BTC, previous day metals	Change in implied volatility / fear	all
dxy_return ‡	Macro	previous-day log return of the US Dollar Index	Metals are priced in USD	Gold, Silver
oil_return ‡	Macro	previous-day log return of WTI crude	Commodity co-movement / inflation proxy	Gold, Silver
tnx_return ‡	Macro	previous-day log return of the 10-year yield	Opportunity cost of holding gold	Gold, Silver
gold_return	Cross-asset	$\ln(GC_t/GC_{t-1})$ same day (gold and silver settle in the same 13:25–13:30 window)	Silver follows gold	Silver
fear_greed	Sentiment	alternative.me index 0–100, published 00:00 UTC for the day	Crypto-specific sentiment	Bitcoin
dow_sin, dow_cos	Calendar	sin / cos(2π·weekday/7)	Weekend effects in a 7-day market	Bitcoin
log_volume_change	Volume	$\text{clip}(\ln(V_t/V_{t-1}), -3, 3)$	Attention / activity spikes	Bitcoin

Table 8. Every model input. † lagged one session for Gold/Silver (Yahoo's futures High/Low extend past the 13:30 settlement); ‡ previous trading day's value for Gold/Silver (those markets close after the settlement). Bitcoin 29 · Gold 28 · Silver 29 features.

Kept for charts only, never model inputs (`config.LEVEL_COLUMNS`): open, high, low, price, volume, EMA_14, EMA_30, BB_Mid, BB_Upper, BB_Lower, ATR, MACD, MACD_Signal. Reason: they are price levels. Gold's test period ($3,986–$5,294) lies entirely above its training range ($1,176–$3,644); a tree model cannot extrapolate a level it has never seen.

Features that exist in the code but are deliberately not used for some assets: futures volume (unusable contract-roll jumps), day-of-week for the metals (no calendar effect after removing synthetic weekends), same-day macro values for the metals (close-time leak). **No feature-selection algorithm** (e.g. RFE) is used; the ablation experiment E2 (validation only) showed that dropping any single group changes walk-forward RMSE by between −0.39 % and +0.05 %, i.e. within noise, so the full set is kept.

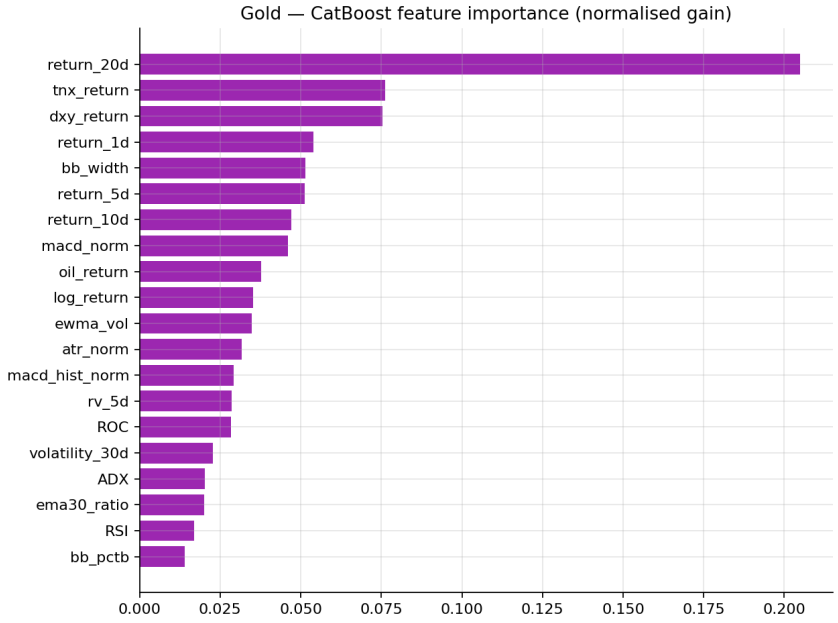


Figure 3. Feature importance of the CV-selected tree model for Gold (CatBoost). GRU/LSTM expose no importance; the combined forecast averages all six.

9. Target Variable

What exactly is predicted. The served task is `return_1d`: the **next trading day's log return**

$$y_t = \ln(P_{t+1} / P_t), \quad \text{predicted price } P'_{t+1} = P_t \cdot \exp(\hat{y}_t), \quad \text{direction} = \text{UP if } \hat{y}_t > 0 \text{ else DOWN}$$

where P′ denotes a predicted price, P_t is the close of the last *complete* bar (day *t*) and P_{t+1} the close of the next row in the asset's own calendar — the next calendar day for Bitcoin, the next exchange day for Gold and Silver (Friday → Monday). The horizon is one bar. The target is created in `build_dataset()` (`_task_target`: `log_return.shift(−1)`) and, for the recurrent models, in `create_sequences()`, which builds windows of 30 rows ending at day *t* and takes the log return of row *t*+1 as the label — the window never contains that row (unit-tested).

Why the return and not the price. Prices are non-stationary (ADF p ≈ 0.97–1.00); a model predicting price levels can reach $R^2 \approx 0.95$ simply by copying today's price, which says nothing about skill. Predicting the return makes the target stationary, makes the random-walk baseline explicit ($\hat{y} = 0$) and makes every comparison honest. The target is standardised with the training mean/std for fitting and inverted at prediction time (`model_status.json: target_mean / target_std`).

Worked example (real row from the Gold test history). Data available up to **2026-09-10** (P_t = $4,364.50) is used to predict the close of **2026-09-11**. The combined predicted return was +0.128 %, i.e. a predicted close of $4,370.09; the actual close was $4,366.20 (actual return +0.039 %), an error of +0.09 % and a direction HIT.

Experimental tasks that exist in the code but are not served: `return_5d` (5-day forward return) and `vol_5d / vol_22d` (forward realised volatility). They were evaluated on validation folds only (experiments E3/E4) and are not part of the dashboard or API.

10. Time-Series Data Splitting

Split	Rule (calendar dates, identical for all assets)	Bitcoin	Gold	Silver	Purpose
Train	target ends on or before 2025-09-10	2,750 samples (2018-03-01 →)	1,874 (2018-03-29 →)	1,874 (2018-03-29 →)	Fit the models
Validation	target after 2025-09-10 and on or before 2026-02-17	160	109	109	Early stopping (phase A), hyper-parameter selection, walk-forward folds
Test (unseen)	target after 2026-02-17	207 (2026-02-18 → 2026-09-12)	143 (2026-02-18 → 2026-09-11)	143 (2026-02-18 → 2026-09-11)	Evaluated ONCE by backtesting.py

Table 9. The frozen chronological split (sample counts = 30-day windows with a valid target; from model_status.json).

Proportions. Bitcoin ≈ 88 / 5 / 7 %; metals ≈ 88 / 5 / 7 % (train / validation / test). The split is by *date*, not by percentage, so all three assets are evaluated on the same calendar period.

Chronological order is strictly preserved; there is no shuffling. A random split would place "tomorrow" in the training set of "today" and, because consecutive days share most of their rolling-window features, would make any model look far better than it is. The split is defined by the dates the *target* covers, so a training target never reaches past the training end date (for multi-day experimental targets this is an exact purge/embargo).

Walk-forward validation inside train+val (`src/evaluation/cross_validation.py`, `TimeSeriesSplit`, 4 folds): the train+validation period is cut into four consecutive validation blocks; for fold *k* the model is trained on everything *before* the block and scored on the block (expanding window). Inside each fold the last 15 % of the training window is the early-stopping monitor. This is how hyper-parameters and the "best single model" are chosen; the test set is never passed to this module. The served combined forecast is scored on the same folds by averaging its members' out-of-fold predictions.

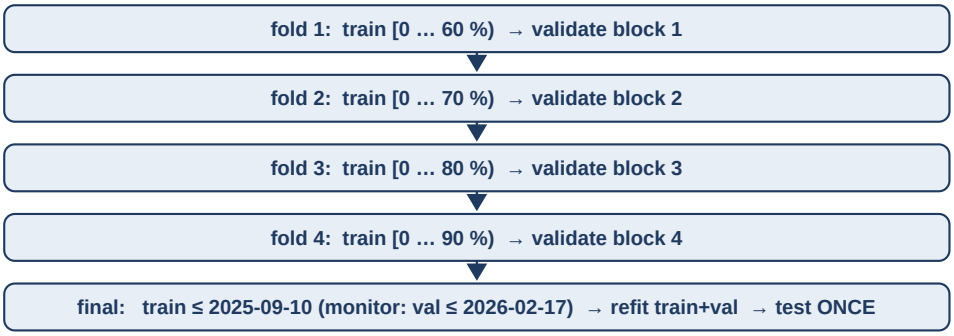


Figure 4. Expanding-window walk-forward folds followed by the single test evaluation.

11. Data Leakage Prevention

Leakage control was the main engineering effort of the project. Each safeguard below is implemented in code and, where marked, enforced by a unit test in `tests/`.

Risk	Safeguard in the implementation	Test
Random shuffling / k-fold	Chronological slicing by frozen dates; TimeSeriesSplit for CV	test_split_is_chronological_and_disjoint
Scaler sees test data	MinMaxScaler.fit(train rows only); saved and reused for val/test/live	test_scaler_fitted_on_train_only
Target inside the input window	label = log return of row t+1; window = rows ≤ t	test_create_sequences_target_is_next_row_and_never_in_window
Rolling / EWM / shift look-ahead	All windows backward-looking; perturbing the last row must not change earlier rows (crypto and futures paths)	test_no_lookahead_in_any_feature
External data alignment	Same-day values only for Bitcoin (bar closes after the US close); latest value strictly before day t for the metals	test_commodity_external_features_are_previous_day_values
Futures High/Low after the settlement	hl_range, atr_norm, ADX shifted one session for the metals	test_commodity_high_low_features_are_lagged_one_session

Risk	Safeguard in the implementation	Test
Synthetic rows	Metals keep the exchange calendar	test_commodities_keep_trading_calendar
Early stopping on its own training data	Two-phase fit: monitor = validation (phase A), refit with fixed stopping point (phase B)	test_tabular_models_fit_predict_and_two_phase
Hyper-parameters chosen on the test set	Tuner reads only train+val (build_dataset splits); best_params.json written from fold scores	by construction (tune_models.py never indexes X_test)
Test set touched more than once	Only backtesting.py reads X_test; before_after.py re-reads stored predictions to compare two frozen systems. The window was also inspected during development (§29), so the pooled walk-forward record (§14.5) and the fresh hold-out (§22) are reported as cleaner evidence	—
Price reconstruction using the wrong column	P_t passed explicitly from the unscaled feature frame	test_true_target_reconstructs_actual_close_exactly
Intraday snapshot treated as a close	Complete-bar rule at download time	test_drop_incomplete_bars_removes_todays_running_row
Combination weights tuned on the test set	Equal weights — nothing is fitted; Combined row = mean of member predictions	test_combined_test_row_is_the_mean_of_its_members
Demo prediction peeking at the future	predict_for_date slices features.loc[:as_of]; every later row is perturbed in the test and the forecast must not change	test_predict_for_date_uses_only_past_data_and_reveals_actual
Band width using future volatility	σ_t = EWMA volatility at day t; k fitted on training-period walk-forward errors only	test_uncertainty_band_is_conditional_calibrated_and_uses_only_day_t_volatility

Table 10. Leakage checklist (55 tests in total pass on the report date).

The leak that was found and fixed (close-time rule). Yahoo's daily close for GC=F / SI=F is the 13:25–13:30 ET COMEX settlement (verified against 5-minute contract bars). The S&P 500 (16:00), VIX (16:15), 10-year yield (~15:00), DXY (17:00) and WTI (14:30) close *after* that, and the futures' daily High/Low span the whole Globex session (until 17:00 ET). An earlier version used their same-day values as day-*t* features for the metals; those values contain 1–3.5 hours of information from *inside* the target interval $\ln(P_{t+1}/P_t)$. That version reported 62.2 % directional accuracy for Silver (DM p = 0.002). After aligning the metals' macro features to the previous trading day and lagging the High/Low-based features one session, the effect disappeared (§15.2). Bitcoin's bar closes at 00:00 UTC, after every US close, so it keeps same-day values. The rule lives in `config.EXTERNAL_SAME_DAY` / `POST_SETTLEMENT_FEATURES` and `DataCleaner._align_external`.

Leakage control in live prediction. The live forecast uses the same feature code on a fresh download that has already had the running bar removed, so the input window ends on the last *complete* bar; the prediction is for the *next* trading day, whose close does not exist yet at prediction time. (The fresh hold-out in §22 scores the unchanged models on closes published after the data were frozen.)

Two small alignment simplifications (documented). Silver uses gold's same-day settlement (13:30 ET) although silver settles at 13:25 ET — a 5-minute overlap at a daily horizon. On weekends Bitcoin sees Friday's S&P 500 / VIX return carried forward — a stale value, not future information. Neither can create a measurable edge, and indeed no edge is measured. A sanity check on the stored features finds no feature whose correlation with the next-day return exceeds 0.07 in absolute value, as expected when nothing leaks.

12. Machine-Learning Models

All models share one interface (`src/models/registry.py`): tabular models receive the 28–29 features of day *t* (the last row of the window); the recurrent models receive the full 30×F window. Every model predicts the same standardised next-day log return, so all are compared on identical samples. Six models are **trained and served** (as members of the combination); five baselines and one stacked ensemble are **evaluated for comparison only**.

Model	Type	Input	Role in the final system	Tuned hyper-parameters (BTC / Gold / Silver)
Naive (random walk)	Baseline	—	$\hat{y} = 0 \Leftrightarrow$ tomorrow's price = today's. The reference every model must beat; DM test partner	—
Historical mean (Naive-Mean)	Baseline	training target	$\hat{y}$ = mean training return (drift)	—
ARIMA(p,0,q)	Baseline (statistical)	return series	Order by AIC on train (max p,q = 3); one-step walk-forward forecasts. Not served	orders: BTC [2, 0, 0], Gold [1, 0, 1], Silver [2, 0, 3]
MA-5	Baseline (fixed rule)	last 5 closes	$\hat{P}$ = mean of the last 5 closes, i.e. $\hat{r} = \ln(MA5/P)$. The classic "naive" moving-average forecast; nothing fitted	—
Mean-20d	Baseline (fixed rule)	last 20 returns	$\hat{r}$ = mean of the last 20 daily log returns; nothing fitted	—
Ridge	Linear (L2-regularised)	day-t features	Member of the Combined forecast	alpha = 100.0 / 100.0 / 100.0
Random Forest	Bagged trees	day-t features	Member	300 trees, max_depth 3, min_samples_leaf 30 (all assets)
LightGBM	Gradient-boosted trees (leaf-wise)	day-t features	Member; early-stopped	lr 0.03 / 0.01 / 0.01 · num_leaves 31 / 31 / 8 · min_child_samples 20 / 100 / 50 · λ 0 / 5 / 5 · subsample 0.8 · colsample 0.8 · up to 600 trees
CatBoost	Gradient-boosted trees (ordered boosting, symmetric)	day-t features	Member; early-stopped; best single model by CV for Bitcoin and Gold	lr 0.03 / 0.01 / 0.01 · depth 7 / 3 / 3 · l2_leaf_reg 3 / 10 / 10 · up to 800 iterations
GRU	Recurrent neural network	30-day window	Member; best single model by CV for Silver	Input → GRU(32) → Dropout(0.4) → Dense(16, relu) → Dense(1); Adam lr 0.001, batch 64, MSE loss, early stopping (patience 6, max 40 epochs)
LSTM	Recurrent neural network	30-day window	Member	Same shape as the GRU with an LSTM(32) cell; dropout 0.4 / 0.2 / 0.2
Stacked ensemble	Meta-model (experiment)	OOF predictions of Ridge, LightGBM, CatBoost, GRU	Reported for comparison only — NOT a member and NOT served	non-negative Ridge(alpha = 1) fitted on walk-forward out-of-fold predictions
Combined	**Equal-weight forecast combination**	**the six members' predicted returns**	**THE SERVED FORECAST (dashboard and API)**	**no parameters — arithmetic mean**

Table 11. Every model in the final system. Hyper-parameters are the values in results/tuning/best_params.json chosen by walk-forward validation.

12.1 Why these models and what each contributes

- **Ridge** — the simplest regularised linear model; a sanity check that any non-linear gain is real. The tuner chose the strongest regularisation on offer (alpha = 100) for all three assets, i.e. it wants to stay close to "no change".
- **Random Forest** — robust non-linear model with little tuning; depth 3 and min_samples_leaf 30 were selected, again the most regularised grid point.
- **LightGBM and CatBoost** — state-of-the-art gradient boosting for tabular data; they can model interactions between volatility state, momentum and macro returns. Early stopping on validation chose few trees (LightGBM 81 / 10 / 10; CatBoost 50 / 606 / 82 for BTC / Gold / Silver), a sign that there is little signal to fit.

- **GRU and LSTM** — sequence models that see the last 30 days of all features rather than one row. They were reduced to a single 32-unit layer because a deeper 2×100-unit stack over-fitted in the original experiments; early stopping selected only 1–9 epochs.
- **Baselines** — random walk, drift and ARIMA make explicit what a model must beat; ARIMA also confirms that linear autocorrelation carries almost no information.
- **Advantages / limitations.** Linear and tree models are fast, interpretable (feature importance) and need no GPU; trees cannot extrapolate beyond training ranges (mitigated by stationary features). Recurrent models can exploit temporal structure but need more data than ~2,000–2,800 windows provide and are the weakest family here. All are point forecasters; the uncertainty band served with the Combined forecast comes from a separate, calibrated volatility rule (§16.2).

13. Model Training

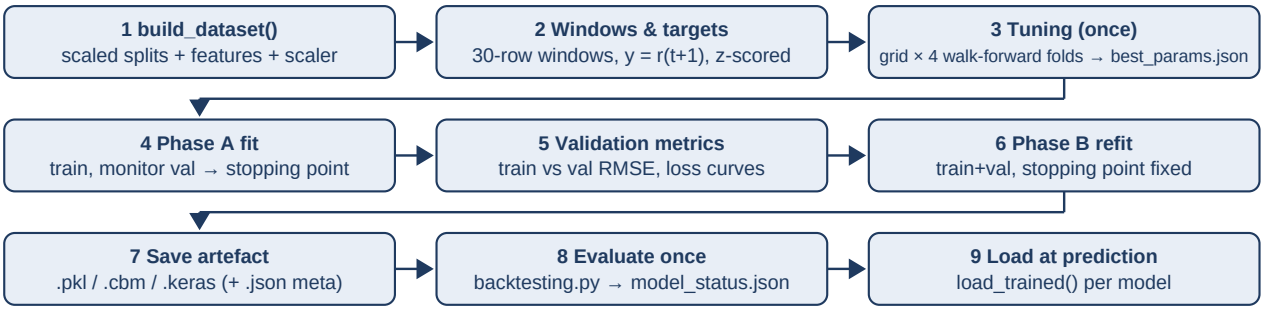


Figure 5. Training pipeline (src/training/tune_models.py → train_models.py → src/evaluation/backtesting.py → src/inference/prediction.py).

1–2 Dataset loading and target creation. `build_dataset(asset)` is the single loader used by tuning, training, stacking, evaluation and the tests. It reads the scaled split files and the unscaled feature frame, builds 30-row windows over the concatenated train|val|test frame (so the first validation/test samples can look back into earlier rows — past data only), attaches the next-row log return as the target, standardises it with training statistics and assigns each window to a split by the date its target covers.

3 Hyper-parameter tuning (`tune_models.py`, `make tune`). Small, deliberate grids over regularisation strength (Ridge 4 configurations, Random Forest 4, LightGBM 48, CatBoost 12, GRU/LSTM 2 each) are scored by mean fold RMSE of the return over the 4 expanding-window folds; every configuration's fold scores are logged to `results/tuning/<asset>_<model>.csv` and the winner to `best_params.json`, which `config.get_params()` reads at training time.

4–6 Two-phase training (`train_models.py`, `make train`). Phase A fits on the training split while monitoring the validation split: LightGBM early-stops with patience 30, CatBoost with 50 rounds, GRU/LSTM with patience 6 on validation loss (restoring the best weights). Phase A yields validation metrics (mildly optimistic for these four models, because they choose their stopping point on the same validation set — model selection therefore uses the walk-forward folds), the train-vs-validation over-fitting gap and the Keras loss curves. Phase B rebuilds the model with the *same* hyper-parameters and the stopping point found in phase A (number of trees / iterations / epochs) and refits on train+validation — this is the deployed model. Validation is therefore never inside its own early-stopping monitor.

7 Model saving. Ridge / Random Forest / LightGBM → `joblib` pickles (model + params + fit info); CatBoost → native `.cbm` + JSON meta; GRU / LSTM → Keras `.keras` + JSON meta with the training history. All 18 artefacts live in `data/models/` next to the three scalers and the stacked meta-models. Reproducibility: seeds 42 for Python, NumPy and TensorFlow; `random_state = 42` for the tree models; pinned requirements.

8–9 Loading during prediction. `src/inference/prediction.py` loads every trained member with `load_trained(name, asset)`, scales the last 30 rows of the (live or frozen) feature frame with the saved scaler, runs each model, de-standardises each predicted return with `target_mean/target_std` from `model_status.json` (an explicit error is raised if these are missing) and averages them.

Asset	Model	Train RMSE (ret)	Val RMSE (ret)	Val / Train	Val dir. acc. %	Stopping point (phase A)
Bitcoin	Ridge	0.03352	0.02605	0.78×	50.6	—

Asset	Model	Train RMSE (ret)	Val RMSE (ret)	Val / Train	Val dir. acc. %	Stopping point (phase A)
Bitcoin	Random Forest	0.03299	0.02610	0.79×	51.2	—
Bitcoin	LightGBM	0.02740	0.02539	0.93×	54.4	81 trees
Bitcoin	CatBoost	0.03209	0.02604	0.81×	44.4	50 iterations
Bitcoin	GRU	0.03361	0.02597	0.77×	53.8	1 epochs
Bitcoin	LSTM	0.03358	0.02617	0.78×	46.2	2 epochs
Gold	Ridge	0.00973	0.02025	2.08×	56.9	—
Gold	Random Forest	0.00953	0.02031	2.13×	54.1	—
Gold	LightGBM	0.00973	0.02028	2.08×	63.3	10 trees
Gold	CatBoost	0.00932	0.01995	2.14×	59.6	606 iterations
Gold	GRU	0.00974	0.02022	2.08×	63.3	8 epochs
Gold	LSTM	0.00975	0.02028	2.08×	63.3	1 epochs
Silver	Ridge	0.01915	0.05543	2.89×	59.6	—
Silver	Random Forest	0.01867	0.05541	2.97×	59.6	—
Silver	LightGBM	0.01919	0.05532	2.88×	62.4	10 trees
Silver	CatBoost	0.01904	0.05528	2.90×	64.2	82 iterations
Silver	GRU	0.01917	0.05528	2.88×	63.3	9 epochs
Silver	LSTM	0.01920	0.05543	2.89×	56.0	4 epochs

Table 12. Phase-A training vs validation metrics (results/train_val_metrics.csv). The validation window (Sep 2025 – Feb 2026) is far more volatile than the 2018–2025 average for the metals, which is why the ratio is > 1 there for every model including the random walk — regime shift, not memorisation. LightGBM, CatBoost, GRU and LSTM choose their stopping point on this validation set, so their validation columns are mildly optimistic.

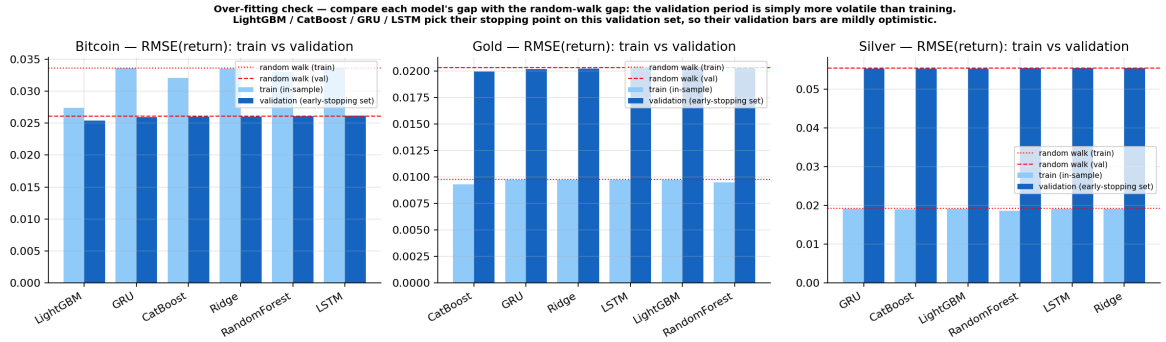


Figure 6. Train vs validation RMSE per model and asset (results/figures/overfitting_gap.png).

14. Model Performance

All numbers in this section are the project's **actual** final results, read from `results/final_test_results.csv` (single evaluation on the untouched test period) and `results/cv_results.csv` (walk-forward validation). Metrics: MAE / RMSE / MAPE in USD (what a user sees); RMSE, MAE and R^2 of the log return (what is actually predicted — $R^2 < 0$ means worse than predicting the mean); directional accuracy on non-flat days with a one-sided binomial p-value against 50 %; Diebold–Mariano p-value against the random walk (squared-error loss, HAC variance, Harvey correction). R^2 on *price levels* is deliberately not reported: a random walk scores ≈ 0.95 there.

14.1 Unseen test set — every model, every asset

Bitcoin — test period 2026-02-18 → 2026-09-12 (207 days)

Model	MAE ($)	RMSE ($)	MAPE %	RMSE (ret)	MAE (ret)	R^2 (ret)	Dir. acc. % (p)	DM p vs RW	Strat. %	B&H %
Naive (random walk)	1,066.42	1,451.73	1.52	0.02079	0.01527	-0.001	—	—	+0.0	+14.5
Historical mean	1,067.32	1,451.38	1.53	0.02078	0.01529	-0.000	49.3 (0.61)	0.81	+14.4	+14.5
ARIMA	1,067.96	1,459.44	1.53	0.02091	0.01530	-0.012	46.9 (0.83)	0.35	-9.0	+14.5
MA-5 (5-day average price)	1,645.82	2,273.75	2.36	0.03265	0.02360	-1.468	52.7 (0.24)	0.00	+20.5	+14.5
Mean-20d (20-day mean return)	1,115.96	1,492.25	1.59	0.02135	0.01597	-0.055	53.1 (0.20)	0.16	+4.5	+14.5
Ridge	1,071.97	1,452.59	1.53	0.02080	0.01535	-0.002	47.8 (0.76)	0.91	+4.4	+14.5
Random Forest	1,064.48	1,448.81	1.52	0.02075	0.01525	+0.004	48.3 (0.71)	0.62	-3.6	+14.5
LightGBM	1,076.49	1,446.18	1.54	0.02068	0.01542	+0.010	51.2 (0.39)	0.79	+5.4	+14.5
CatBoost (CV-best)	1,071.07	1,453.76	1.53	0.02082	0.01534	-0.004	46.9 (0.83)	0.74	-5.6	+14.5
GRU	1,075.86	1,458.31	1.54	0.02089	0.01541	-0.010	46.9 (0.83)	0.35	-14.0	+14.5
LSTM	1,068.06	1,453.44	1.53	0.02082	0.01530	-0.004	46.4 (0.87)	0.35	-7.8	+14.5
Stacked ensemble (experiment)	1,061.35	1,442.94	1.52	0.02065	0.01520	+0.013	50.2 (0.50)	0.44	+2.3	+14.5
Combined (served)	**1,063.94**	**1,447.37**	**1.52**	**0.02072**	**0.01524**	**+0.006**	**46.9 (0.83)**	**0.49**	**-5.1**	**+14.5**

Gold — test period 2026-02-18 → 2026-09-11 (143 days)

Model	MAE ($)	RMSE ($)	MAPE %	RMSE (ret)	MAE (ret)	R^2 (ret)	Dir. acc. % (p)	DM p vs RW	Strat. %	B&H %
Naive (random walk)	58.26	75.76	1.29	0.01671	0.01290	-0.002	—	—	+0.0	-10.6
Historical mean	58.39	75.99	1.30	0.01675	0.01292	-0.007	49.7 (0.57)	0.43	-10.7	-10.6
ARIMA	58.57	76.27	1.30	0.01681	0.01296	-0.015	50.3 (0.50)	0.52	-5.8	-10.6
MA-5 (5-day average price)	85.73	115.37	1.91	0.02546	0.01896	-1.325	50.3 (0.50)	0.00	-14.1	-10.6
Mean-20d (20-day mean return)	61.46	78.96	1.36	0.01741	0.01361	-0.087	46.9 (0.80)	0.04	-8.6	-10.6
Ridge	58.51	76.05	1.30	0.01677	0.01294	-0.009	50.3 (0.50)	0.76	-7.6	-10.6
Random Forest	58.08	75.10	1.29	0.01659	0.01286	+0.012	49.0 (0.63)	0.57	-10.4	-10.6
LightGBM	58.25	75.81	1.29	0.01672	0.01289	-0.003	49.7 (0.57)	0.88	-10.7	-10.6
CatBoost (CV-best)	58.43	76.22	1.30	0.01681	0.01293	-0.015	51.7 (0.37)	0.71	-7.0	-10.6
GRU	58.00	75.42	1.29	0.01664	0.01284	+0.007	51.7 (0.37)	0.58	-6.5	-10.6
LSTM	58.82	76.58	1.31	0.01687	0.01301	-0.022	49.7 (0.57)	0.28	-10.7	-10.6
Stacked ensemble (experiment)	58.95	77.28	1.31	0.01705	0.01303	-0.043	51.0 (0.43)	0.44	-7.9	-10.6
Combined (served)	**58.20**	**75.73**	**1.29**	**0.01671**	**0.01288**	**-0.001**	**50.3 (0.50)**	**0.97**	**-7.9**	**-10.6**

Silver — test period 2026-02-18 → 2026-09-11 (143 days)

Model	MAE ($)	RMSE ($)	MAPE %	RMSE (ret)	MAE (ret)	R² (ret)	Dir. acc. % (p)	DM p vs RW	Strat. %	B&H %
Naive (random walk)	1.79	2.37	2.49	0.03189	0.02479	-0.001	—	—	+0.0	-12.1
Historical mean	1.79	2.38	2.49	0.03192	0.02477	-0.003	51.7 (0.37)	0.63	-12.2	-12.1
ARIMA	1.81	2.37	2.52	0.03226	0.02515	-0.024	52.4 (0.31)	0.50	-3.2	-12.1
MA-5 (5-day average price)	2.80	3.67	3.89	0.04958	0.03862	-1.418	52.4 (0.31)	0.00	-18.5	-12.1
Mean-20d (20-day mean return)	1.93	2.51	2.67	0.03372	0.02666	-0.119	42.7 (0.97)	0.00	-22.7	-12.1
Ridge	1.79	2.37	2.48	0.03181	0.02466	+0.005	54.5 (0.16)	0.48	-0.7	-12.1
Random Forest	1.76	2.33	2.44	0.03146	0.02433	+0.026	56.6 (0.07)	0.17	+6.5	-12.1
LightGBM	1.79	2.37	2.48	0.03189	0.02466	-0.001	58.7 (0.02)	0.98	+10.1	-12.1
CatBoost	1.79	2.38	2.49	0.03193	0.02477	-0.003	51.7 (0.37)	0.70	-12.2	-12.1
GRU (CV-best)	1.79	2.37	2.48	0.03189	0.02468	-0.001	55.2 (0.12)	0.99	+3.7	-12.1
LSTM	1.80	2.38	2.49	0.03198	0.02484	-0.006	51.7 (0.37)	0.63	-12.2	-12.1
Stacked ensemble (experiment)	1.79	2.38	2.48	0.03191	0.02465	-0.002	52.4 (0.31)	0.91	-11.0	-12.1
Combined (served)	**1.78**	**2.37**	**2.47**	**0.03180**	**0.02463**	**+0.005**	**53.8 (0.20)**	**0.46**	**-6.0**	**-12.1**

Table 13 (a–c). Untouched test set. Shaded row = the served Combined forecast; (CV-best) = best single model by walk-forward validation. "Strat." = cumulative return of a long/flat rule (hold the asset on day t+1 iff predicted return > 0, 10 bps per position change); "B&H" (buy & hold) = the asset's own return over the period.

14.2 Served forecast vs random walk — summary

Asset	n days	MAE ($) Combined / RW	RMSE ($) Combined / RW	MAPE % Combined / RW	RMSE (ret) Combined / RW	Δ RMSE (ret)	Direction correct (p)	DM p
Bitcoin	207	1,063.94 / 1,066.42	1,447.37 / 1,451.73	1.52 / 1.52	0.02072 / 0.02079	-0.34 %	46.9 % (0.83)	0.49
Gold	143	58.20 / 58.26	75.73 / 75.76	1.29 / 1.29	0.01671 / 0.01671	-0.04 %	50.3 % (0.50)	0.97
Silver	143	1.78 / 1.79	2.37 / 2.37	2.47 / 2.49	0.03180 / 0.03189	-0.29 %	53.8 % (0.20)	0.46

Table 14. The served Combined forecast against the random walk (RW) on the unseen test period.

Reading. For all three assets every model sits at the random-walk floor. The combined forecast's return-RMSE is 0.3 % below the random walk for Bitcoin and Silver and equal for Gold; none of the differences is significant (DM p = 0.49 / 0.97 / 0.46). R^2 in return space is ≈ 0 for every model. Directional accuracy of the combined forecast is 46.9 / 50.3 / 53.8 % (p = 0.83 / 0.50 / 0.20). The dollar errors (MAPE 1.52 / 1.29 / 2.47 %) equal the random walk's because they are the daily volatility of each asset, not model skill — **MAPE must not be read as "accuracy"**. The models have learned that tomorrow's move is mostly unpredictable: the standard deviation of the combined prediction is 0.12 % (Bitcoin), 0.09 % (Gold) and 0.09 % (Silver) against realised return standard deviations of 2.08 / 1.67 / 3.19 %.

The one p < 0.05 cell. Silver LightGBM reaches 58.7 % directional accuracy (binomial p = 0.02) but has exactly the random walk's RMSE (DM p = 0.98); with 36 model-rows in the test tables one cell at p ≈ 0.02 is what chance produces (Bonferroni threshold ≈ 0.0014), its walk-forward figure is 53.3 %, and it was not the validation winner. It is reported, not claimed.

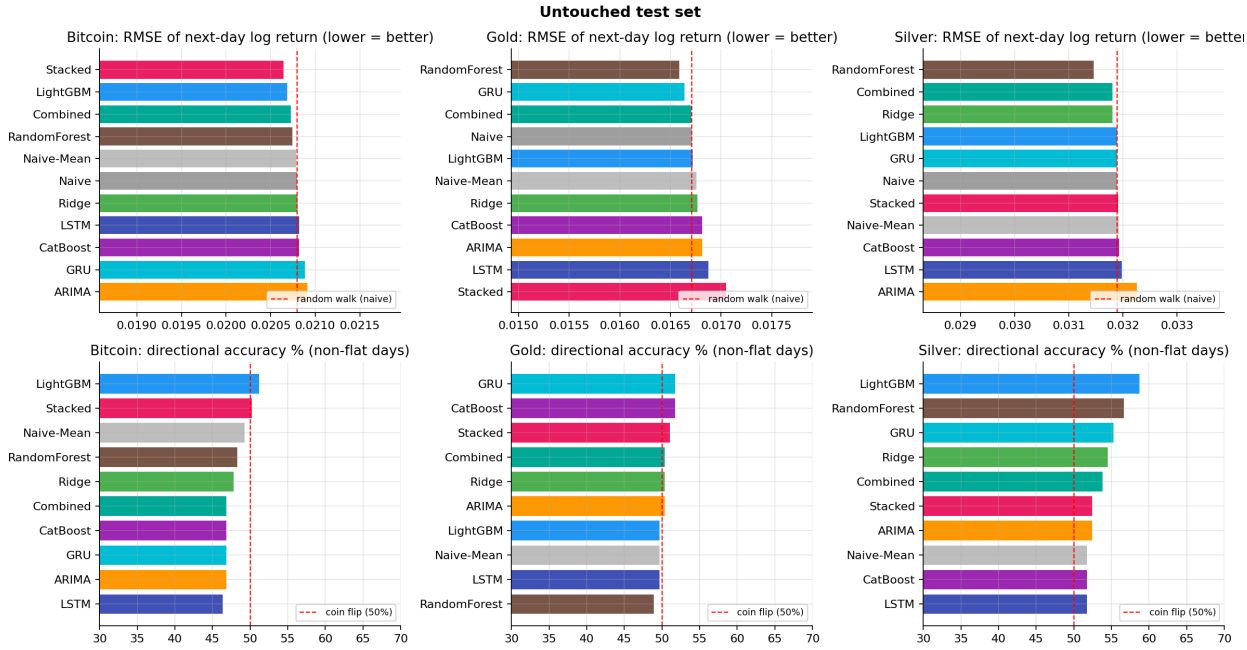


Figure 7. Test-set RMSE of the next-day return and directional accuracy for every model and asset (results/figures/model_comparison_test.png). The RMSE axes are zoomed: all models lie within ±1 % of the random walk.

14.3 Walk-forward validation (model-selection evidence)

Bitcoin — 4 expanding folds inside train+validation

Model	RMSE (ret) mean ± std	MAE (ret)	R² (ret)	Dir. acc. %	RMSE ($)
Naive (random walk)	0.03160 ± 0.00690	0.02130	-0.005	—	1,429.08
Historical mean	0.03163 ± 0.00694	0.02133	-0.007	48.8	1,430.95
ARIMA	0.03164 ± 0.00695	0.02135	-0.007	48.1	1,431.80
MA-5 (5-day average price)	0.04607 ± 0.01001	0.03299	-1.138	52.1	2,059.25
Mean-20d (20-day mean return)	0.03232 ± 0.00702	0.02219	-0.052	50.3	1,466.63
Ridge	0.03161 ± 0.00693	0.02135	-0.005	49.4	1,430.34
Random Forest	0.03160 ± 0.00688	0.02140	-0.005	50.0	1,432.90
LightGBM	0.03158 ± 0.00692	0.02140	-0.003	51.2	1,430.52
CatBoost (CV-best)	0.03156 ± 0.00695	0.02136	-0.001	50.2	1,427.07
GRU	0.03168 ± 0.00697	0.02136	-0.009	49.4	1,432.61
LSTM	0.03170 ± 0.00697	0.02142	-0.010	48.8	1,433.09
Combined (served)	**0.03159 ± 0.00693**	**0.02133**	**-0.004**	**50.8**	**1,429.51**

Gold — 4 expanding folds inside train+validation

Model	RMSE (ret) mean ± std	MAE (ret)	R² (ret)	Dir. acc. %	RMSE ($)
Naive (random walk)	0.01114 ± 0.00225	0.00790	-0.006	—	28.46
Historical mean	0.01113 ± 0.00223	0.00787	-0.004	55.8	28.43
ARIMA	0.01123 ± 0.00219	0.00796	-0.025	50.5	28.64
MA-5 (5-day average price)	0.01595 ± 0.00296	0.01195	-1.075	50.1	39.43
Mean-20d (20-day mean return)	0.01147 ± 0.00226	0.00806	-0.068	51.1	29.28
Ridge	0.01112 ± 0.00223	0.00787	-0.003	54.8	28.40
Random Forest	0.01116 ± 0.00220	0.00791	-0.012	51.9	28.45
LightGBM	0.01113 ± 0.00223	0.00787	-0.004	55.8	28.42
CatBoost (CV-best)	0.01110 ± 0.00219	0.00786	-0.001	54.8	28.29
GRU	0.01114 ± 0.00222	0.00791	-0.007	51.1	28.45
LSTM	0.01116 ± 0.00223	0.00793	-0.010	47.0	28.49
Combined (served)	**0.01113 ± 0.00222**	**0.00788**	**-0.004**	**52.5**	**28.39**

Silver — 4 expanding folds inside train+validation

Model	RMSE (ret) mean ± std	MAE (ret)	R² (ret)	Dir. acc. %	RMSE ($)
Naive (random walk)	0.02391 ± 0.00624	0.01596	-0.002	—	1.00
Historical mean	0.02391 ± 0.00623	0.01594	-0.002	53.5	1.00
ARIMA	0.02410 ± 0.00613	0.01619	-0.021	50.2	1.00
MA-5 (5-day average price)	0.03468 ± 0.00776	0.02380	-1.152	52.6	1.33
Mean-20d (20-day mean return)	0.02453 ± 0.00622	0.01634	-0.059	49.6	1.03
Ridge	0.02393 ± 0.00625	0.01593	-0.003	53.3	1.00
Random Forest	0.02396 ± 0.00619	0.01596	-0.007	51.1	1.01
LightGBM	0.02389 ± 0.00625	0.01592	-0.000	53.3	1.00
CatBoost	0.02391 ± 0.00623	0.01594	-0.002	53.5	1.00
GRU (CV-best)	0.02387 ± 0.00625	0.01592	+0.002	52.3	1.00
LSTM	0.02393 ± 0.00625	0.01595	-0.003	51.2	1.00
Combined (served)	**0.02390 ± 0.00624**	**0.01592**	**-0.001**	**52.7**	**1.00**

Table 15 (a–c). Walk-forward validation. All trained models lie within ~0.5 % of each other and of the random walk (the moving-average rules are far worse), so the ranking is not statistically decisive — the reason the served forecast is the plain equal-weight combination rather than a single "winner".

14.4 Performance across market regimes (test period)

Asset	Regime (defined on day t, before the prediction)	Days	MAE % Combined	MAE % random walk	Direction correct %
Bitcoin	high volatility	69	1.65	1.66	50.7
Bitcoin	low volatility	69	1.30	1.29	47.8
Bitcoin	medium volatility	69	1.62	1.62	42.0
Bitcoin	down-trend (20d)	96	1.62	1.62	47.9
Bitcoin	up-trend (20d)	111	1.43	1.44	45.9
Gold	high volatility	48	1.34	1.35	50.0
Gold	low volatility	48	1.27	1.27	50.0
Gold	medium volatility	47	1.26	1.26	51.1
Gold	down-trend (20d)	86	1.35	1.35	52.3
Gold	up-trend (20d)	57	1.21	1.21	47.4
Silver	high volatility	48	2.92	2.93	52.1
Silver	low volatility	48	2.11	2.14	64.6
Silver	medium volatility	47	2.39	2.38	44.7
Silver	down-trend (20d)	81	2.55	2.58	58.0
Silver	up-trend (20d)	62	2.37	2.36	48.4

Table 16. results/regime_analysis.csv — volatility terciles of the 30-day σ and sign of the 20-day return. Errors scale with volatility for every asset, and the combined forecast neither improves nor degrades relative to the random walk in any regime.

14.5 Long-run evidence — ≈ 6 years of pooled walk-forward predictions

The test window has only 143–207 days, too few to detect a small edge. `backtesting.py :: pooled_walkforward` therefore pools every out-of-fold prediction of the four walk-forward folds — about six years per asset, train+validation only — and tests each forecast against the random walk on all of them at once. Hyper-parameters were tuned on these folds, so any bias favours the models. A direction hit-rate must also beat the drift forecast ("always UP", the historical mean), not just 50 %, to show timing skill.

Asset	Days	Forecast	RMSE vs random walk	DM p vs RW	DM p vs drift	Direction % (p vs 50 %)	UP calls % / up days %	corr($\hat{r}$, r)
Bitcoin	2,328	Combined (served)	-0.018 %	0.866	0.090	50.8 (0.234)	67 / 51	+0.018
Bitcoin	2,328	Historical mean	+0.134 %	0.025	—	48.8 (0.881)	75 / 51	-0.064
Bitcoin	2,328	CatBoost	-0.091 %	0.448	0.039	50.2 (0.426)	68 / 51	+0.038
Bitcoin	2,328	LightGBM	-0.034 %	0.862	0.363	51.2 (0.136)	58 / 51	+0.039
Bitcoin	2,328	Ridge	+0.050 %	0.682	0.490	49.4 (0.740)	60 / 51	+0.008
Bitcoin	2,328	MA-5 (5-day average price)	+45.752 %	0.000	0.000	52.1 (0.022)	47 / 51	+0.015

Asset	Days	Forecast	RMSE vs random walk	DM p vs RW	DM p vs drift	Direction % (p vs 50 %)	UP calls % / up days %	corr(r̂, r)
Gold	1,584	Combined (served)	-0.213 %	0.085	0.570	52.5 (0.023)	74 / 56	+0.048
Gold	1,584	Historical mean	-0.142 %	0.070	—	55.8 (0.000)	100 / 56	-0.017
Gold	1,584	CatBoost	-0.436 %	0.283	0.488	54.8 (0.000)	87 / 56	+0.079
Gold	1,584	LightGBM	-0.147 %	0.061	0.900	55.8 (0.000)	100 / 56	-0.002
Gold	1,584	Ridge	-0.208 %	0.050	0.410	54.8 (0.000)	90 / 56	+0.033
Gold	1,584	MA-5 (5-day average price)	+42.653 %	0.000	0.000	50.1 (0.470)	44 / 56	+0.051
Silver	1,584	Combined (served)	-0.058 %	0.691	0.805	52.7 (0.016)	78 / 53	+0.018
Silver	1,584	Historical mean	-0.027 %	0.507	—	53.5 (0.003)	100 / 53	-0.025
Silver	1,584	CatBoost	-0.031 %	0.612	0.901	53.5 (0.003)	94 / 53	-0.006
Silver	1,584	LightGBM	-0.072 %	0.416	0.547	53.3 (0.005)	83 / 53	+0.025
Silver	1,584	Ridge	+0.053 %	0.737	0.564	53.3 (0.005)	78 / 53	-0.027
Silver	1,584	MA-5 (5-day average price)	+43.779 %	0.000	0.000	52.6 (0.021)	46 / 53	+0.034

Table 16b. results/walkforward_pooled.csv (selected rows; every model is in the file). Combined vs random walk: no significant difference for any asset; the metals' 52–53 % direction hit-rates are below the drift forecast's, so they reflect the rising average, not timing.

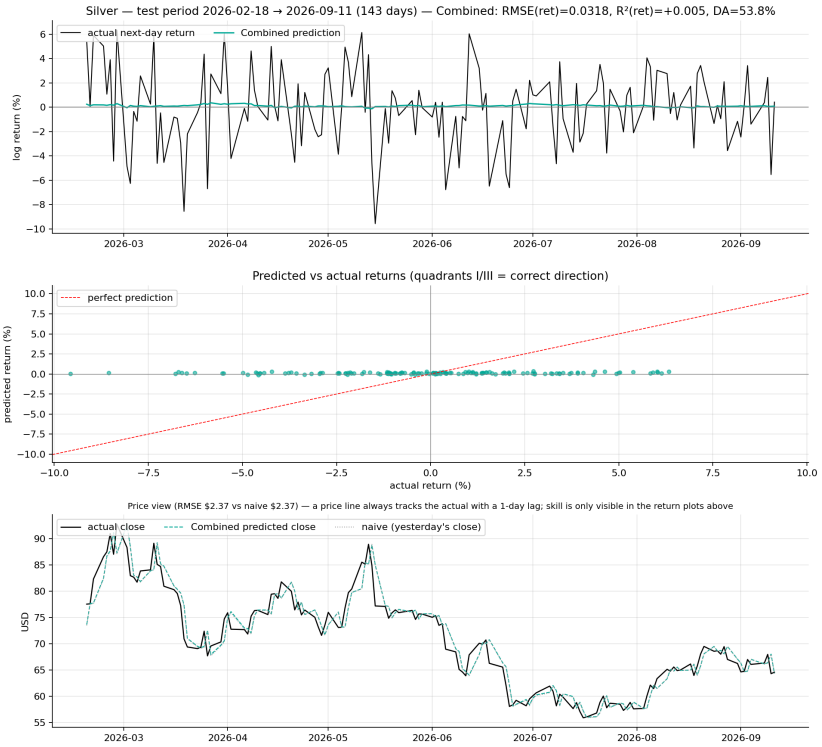


Figure 8. Silver, unseen test period: predicted vs actual next-day returns (line and scatter) and the reconstructed price with the random-walk shadow (results/figures/silver_actual_vs_predicted.png).

15. Baseline Comparison

Yes — the project is built around a baseline. The primary baseline is the **random walk** (`NaiveModel`): the predicted log return is 0, i.e. "the next price equals the latest known price". Four further baselines are evaluated on the same days: the **historical mean** (drift), an **ARIMA(p,0,q)** on the return series (orders chosen by AIC on the training window: Bitcoin (2, 0, 0), Gold (1, 0, 1), Silver (2, 0, 3); one-step walk-forward forecasts) and two fixed **moving-average rules** — **MA-5** (next close = mean of the last five closes) and **Mean-20d** (next return = mean of the last 20 returns) — the "naive" benchmarks often used in forecasting projects. Nothing is fitted for the two rules.

Why it matters. In dollar terms a random walk already achieves MAPE 1.3–2.5 % and price-level $R^2 \approx 0.95$, so a model can look impressive while adding nothing. The project therefore (i) reports every metric side by side with the naive forecast, (ii) tests the difference formally with the Diebold–Mariano statistic, and (iii) tests directional accuracy against 50 % with a binomial test. The random walk is also the dashboard's reference line in every chart

and table.

Result of the comparison. Table 14 gives the answer for the served forecast: within ±0.3 % of the random-walk RMSE, DM p ≥ 0.46, direction not distinguishable from chance. The best single models are 0.0–0.6 % *above* the random walk's RMSE on the test set. The historical-mean and ARIMA baselines are likewise indistinguishable from the random walk. The moving-average rules are **52.3–57.0 % worse** than the random walk (MA-5) and 2.7–5.7 % worse (Mean-20d): beating a moving average says nothing — the random walk is the benchmark that matters.

15.1 Before / after comparison on identical unseen days

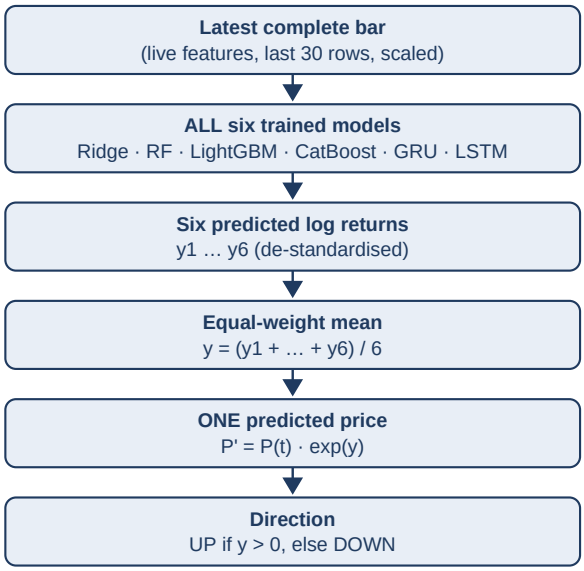
Asset	System	Model	MAE ($)	RMSE ($)	MAPE %	RMSE (ret)	R² (ret)	Dir. acc. % (p)	DM p
Bitcoin	naive (old data)	Naive	1,093.48	1,421.12	1.58	0.02064	-0.000	—	—
Bitcoin	before (3 y data)	CatBoost	1,096.70	1,424.81	1.59	0.02068	-0.005	47.8 (0.74)	0.57
Bitcoin	naive (new data)	Naive	1,092.36	1,420.93	1.58	0.02064	-0.000	—	—
Bitcoin	after (full history)	CatBoost	1,100.64	1,425.57	1.59	0.02071	-0.007	46.6 (0.83)	0.34
Gold	naive (old data)	Naive	60.97	78.84	1.34	0.01726	-0.010	—	—
Gold	before (3 y data)	CatBoost	61.74	80.03	1.36	0.01752	-0.041	47.7 (0.72)	0.15
Gold	naive (new data)	Naive	60.87	78.79	1.34	0.01725	-0.010	—	—
Gold	after (full history)	CatBoost	61.07	79.54	1.35	0.01742	-0.030	49.5 (0.58)	0.62
Silver	naive (old data)	Naive	1.95	2.57	2.65	0.03397	-0.004	—	—
Silver	before (3 y data)	CatBoost	1.97	2.59	2.67	0.03418	-0.017	48.6 (0.65)	0.27
Silver	naive (new data)	Naive	1.96	2.57	2.66	0.03399	-0.004	—	—
Silver	after (full history)	GRU	1.95	2.57	2.65	0.03403	-0.007	53.2 (0.28)	0.74

Table 17. results/experiments/before_after.csv — the archived 3-year system vs the final full-history system on the same 161 / 111 unseen days (2026-02-18 → 2026-07-28). 2.9× more training data moved the single models onto the random-walk line but did not produce skill.

15.2 The archived "62 % Silver" result

The first evaluation of the improved system (kept in `results/archive_sameday_macro_leak/`) reported Silver LightGBM at **62.2 % directional accuracy, R² = +0.044, DM p = 0.002**. It used same-day macro returns for the metals. After the close-time correction (§11) the same model on the same 143 days scores 51.7 % with the five macro features removed and the final served single model (GRU) 55.2 % (p = 0.12, DM p = 0.99). Bitcoin's numbers are bit-identical before and after (its bar closes after the US close). The lesson written into the methodology: "backward-looking by date" is not the same as "known at the moment the target interval starts".

16. Ensemble / Combined Prediction — the served forecast



Exact rule (`prediction.py :: _predict_from_window`): each trained member predicts the standardised return from the same input window; each value is mapped back to a real log return with the training mean/std; the served return is the **arithmetic mean** of the members' returns — no weights are fitted, no voting, no optimisation. Price and direction follow from the mean. The stacked ensemble is *not* a member (it is itself a blend of some members).

Why an equal-weight combination. (i) All six members are within ~0.5 % of each other on the walk-forward folds, so picking one "winner" would be picking noise. (ii) Equal weights are the standard robust choice for forecasts of similar quality — estimated weights rarely beat them out of sample (the forecast-combination puzzle). (iii) Nothing is fitted, so the combination cannot be tuned on the test set. (iv) The user never has to choose a model.

Evaluated like a single model. The combination is scored on the walk-forward folds (members' out-of-fold predictions averaged per fold, row "Combined" in Table 15) and once on the test set (row "Combined" in Table 13); a unit test asserts that the stored Combined test predictions equal the mean of the stored member predictions.

16.1 The stacked ensemble (experiment, not served)

Asset	Base models	Fitted non-negative weights	Intercept	Test RMSE (ret) stacked vs Combined
Bitcoin	Ridge, LightGBM, CatBoost, GRU	Ridge 0.00, LightGBM 0.39, CatBoost 0.31, GRU 0.00	0.006	0.02065 vs 0.02072
Gold	Ridge, LightGBM, CatBoost, GRU	Ridge 0.08, LightGBM 0.00, CatBoost 1.18, GRU 0.59	0.083	0.01705 vs 0.01671
Silver	Ridge, LightGBM, CatBoost, GRU	Ridge 0.00, LightGBM 0.55, CatBoost 0.00, GRU 0.99	0.021	0.03191 vs 0.03180

Table 18. results/stacking/.json and the test rows. Weights are fitted on walk-forward out-of-fold predictions (train+val only). The stack is marginally better than the equal-weight combination for Bitcoin and worse for Gold and Silver — no consistent advantage, so it stays an experiment.*

Why the combined prediction is useful even without skill. It gives one unambiguous answer instead of six, averaging damps the idiosyncratic noise of individual members (the combined prediction's spread is 0.09–0.12 % of price against realised daily moves of 1.7–3.2 %), and it removes the temptation to cherry-pick the model that happened to do best on the test window.

16.2 The calibrated 68 % uncertainty band

Every served forecast carries a band $\mathbf{P_t \cdot \exp(\hat{r} \pm k \cdot \sigma_t)}$, where σ_t is the RiskMetrics EWMA volatility (λ = 0.94) of the returns up to day t (the `ewma_vol` feature) and k is calibrated once by `backtesting.py :: band_rows` as the 68.27 % quantile of $|r - \hat{r}| / \sigma_t$ over the walk-forward out-of-fold errors of the **training** period. The band therefore widens in volatile regimes and narrows in calm ones — volatility is the one quantity this data does predict (experiment E4) — and its coverage on validation and test is out-of-sample. Daily returns are fat-tailed, so a plain ±1σ band over-covers; k < 1 corrects that. It replaced the earlier ±1-RMSE band, whose width came from the test period itself.

Asset	k	Split	Served band k·σ_t: coverage %	Plain ±1σ_t: coverage %	Fixed width: coverage %	Mean half-width %
Bitcoin	0.83	validation	60.6	75.0	79.4	1.80
Bitcoin	0.83	test	65.7	73.9	79.2	1.82
Gold	0.91	validation	66.1	72.5	47.7	1.43
Gold	0.91	test	69.2	75.5	42.0	1.68
Silver	0.87	validation	54.1	57.8	42.2	3.22
Silver	0.87	test	69.2	75.5	44.8	3.47

Table 18b. results/band_calibration.csv — share of days whose actual close fell inside each band (nominal 68.3 %). On the test period the served band is close to nominal for all three assets; on the shorter, more volatile validation window it under-covers for Bitcoin and Silver (Silver's late-2025 volatility jump makes every EWMA band too narrow there).

17. How a User Generates a Prediction

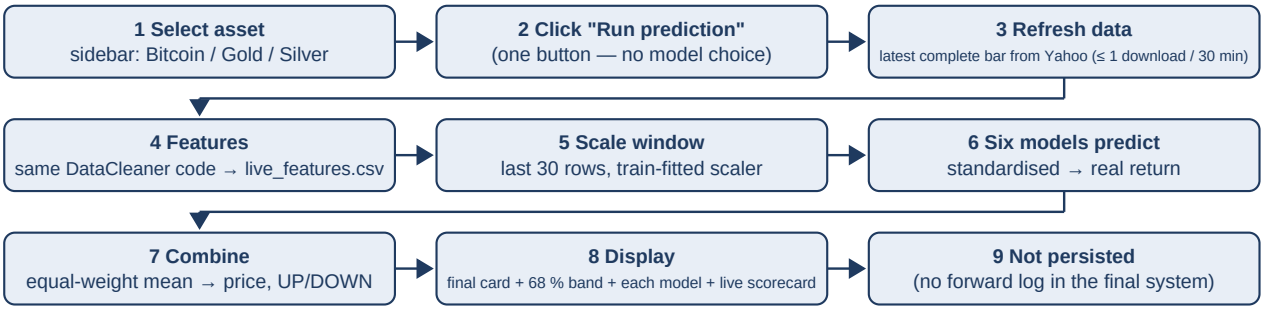


Figure 9. The dashboard "Run prediction" workflow (app/streamlit_app.py → src/data/sync_live_data.py → src/inference/prediction.py).

Step details. (3) `update_live_data()` downloads the asset, the external series and — for Silver — gold into `data/raw_live/` (the frozen `data/raw/` is never touched), unless the live file already ends on the newest complete bar or was refreshed less than 30 minutes ago. (4) The same `DataCleaner` builds the features and writes `<prefix>_live_features.csv` atomically; a download that fails feature validation is discarded. (5–7) `predict_next_day(asset)` as described in §16; the target date is the next calendar day (Bitcoin) or the next weekday (metals). (8) The final card shows the predicted close, UP/DOWN, the change in % and $, and the calibrated 68 % band (§16.2); below it the live scorecard — each member and the Combined forecast with tomorrow's prediction and its direction hits and average error against the random walk on the last 20–90 days, re-computed on every press — a 60-day chart with the forecast point and the band, and the reliability table (Combined vs random walk on the unseen test days). If Yahoo has not yet published the newest bar, a warning names the bar the forecast starts from; if the download itself fails (offline or rate-limited), the warning says so and names the stored dataset and its last bar. (9) The forecast is displayed, not stored (see §19).

17.1 Example run (actual output on 13 November 2026 — illustrative only, not a result)

The following was produced by `predict_next_day()` — the function behind `make predict`, the dashboard button and the API — while this report was built. It illustrates the mechanics; it is not evidence of accuracy.

```
Bitcoin  last complete bar 2026-11-12 (live download), close $71,342.97 → predicted close for
2026-11-13: $71,285.44 (-0.08 %, DOWN)
members: Ridge -0.18 % · RandomForest -0.01 % · LightGBM +0.16 % · CatBoost -0.28 % · GRU -0.13 % ·
LSTM -0.04 % → mean -0.08 %
68 % band $70,047.12 – $72,545.65 (±1.75 % = 0.83 × σ_t 2.12 %)

Gold  last complete bar 2026-11-12 (live download), close $3,885.30 → predicted close for 2026-11-13:
$3,890.81 (+0.14 %, UP)
members: Ridge +0.20 % · RandomForest +0.16 % · LightGBM +0.06 % · CatBoost +0.22 % · GRU +0.04 % ·
LSTM +0.18 % → mean +0.14 %
68 % band $3,834.98 – $3,947.46 (±1.45 % = 0.91 × σ_t 1.59 %)

Silver  last complete bar 2026-11-12 (live download), close $49.97 → predicted close for 2026-11-13:
$50.05 (+0.15 %, UP)
members: Ridge +0.22 % · RandomForest +0.24 % · LightGBM +0.10 % · CatBoost +0.10 % · GRU +0.17 % ·
LSTM +0.06 % → mean +0.15 %
68 % band $48.90 – $51.22 (±2.32 % = 0.87 × σ_t 2.67 %)
```

Note how the members agree to within a fraction of a percent and how the band (±1.4 % for Gold) is far wider than the predicted move (+0.14 %): this is the honest picture of next-day uncertainty, and the dashboard says so next to the number.

18. UP / DOWN Prediction

The direction is derived from the **sign of the combined predicted log return**, which is equivalent to comparing the predicted price with the latest known close:

```
direction = "UP" if ŷ > 0 else "DOWN"    (⇔ P′_{t+1} > P_t → UP, otherwise DOWN)    — prediction.py :: predict_next_day / predict_for_date / _individual
```

There is no dead-band or threshold: an exactly-zero predicted return (which does not occur with continuous models) would be labelled DOWN. Each member model gets its own UP/DOWN label by the same rule, and the dashboard reports how many of the six members agree (e.g. "5 of 6 models say UP"), but the served direction is the sign of the *mean*, not a majority vote. For the statistics, "directional accuracy" counts a day as correct when sign(ŷ) = sign(actual r) among days whose actual return is non-zero (§20).

19. Automatic Daily Prediction System — status in the final implementation

Status: not part of the final system. During the final audit (2026-09-13/14) an automatic daily forward-prediction record was implemented — a scheduled job that ran the combined forecast for each asset after the day's bar was complete, appended it to an append-only log, and on the following run fetched the actual close, computed the error and a HIT/FAIL label, and showed the record in a dashboard tab. It was verified end-to-end and then **removed entirely at the author's request** on 14 September 2026 ("the project keeps the frozen test-set history as its evidence of out-of-sample prediction"). The final code base contains **no scheduler, no launchd/cron job and no persistent forward-prediction file**, and the dashboard has no "automatic predictions" tab. The screenshots in this report were taken from the final code.

What the final system does instead — and how it maps onto the Day-1 / Day-2 idea. The *logic* of forward verification is fully present; what is missing is only the persistence and the scheduler:

Element of an automatic daily system	Present in the final system?	Where / how
Fetch the latest available data automatically	Yes, on demand	"Run prediction" (dashboard) or GET /predict/{asset} (API) refreshes the last complete bar (throttled)
Run all models and generate the combined prediction	Yes	predict_next_day() — six models, equal-weight mean
Save the prediction before the target close exists	No (removed)	The live forecast is displayed/returned, not written to disk
Next day: fetch the actual price and compare with the stored prediction	No live log; Yes on the frozen test period, on the recent live days and on the post-freeze days	predict_for_date() and results/predictions/*.csv (every unseen test day); recent_track_record() (live scorecard, last 20–90 days); src/experiments/fresh_holdout.py (§22)
Calculate error and HIT/FAIL	Yes	error_usd, error_pct, direction_hit (§20)
Maintain a growing history	No persistent live history; a fixed history of 143–207 unseen days per asset, and a Predict-a-Day list that grows with every newly published close	results/predictions/*.csv; Predict-a-Day (days marked NEW)
Scheduler	No	—

Table 19. Automatic-prediction elements versus the final implementation.

The out-of-sample verification loop that IS implemented (frozen test period). For every day *t* of the unseen period the evaluation script stands on day *t*, builds the input window from data up to *t*, produces every model's forecast for *t*+1, then reads the actual close of *t*+1 and records error and direction hit. This is exactly the Day-1 → Day-2 comparison, carried out once for all test days from the frozen dataset (the models were trained only on data

up to the validation end date, so each of these days is genuinely unseen). The dashboard exposes the same loop interactively (Predict a Day — Figure 10) and re-runs it on the most recent days in the live scorecard; the per-day files are described in §21.

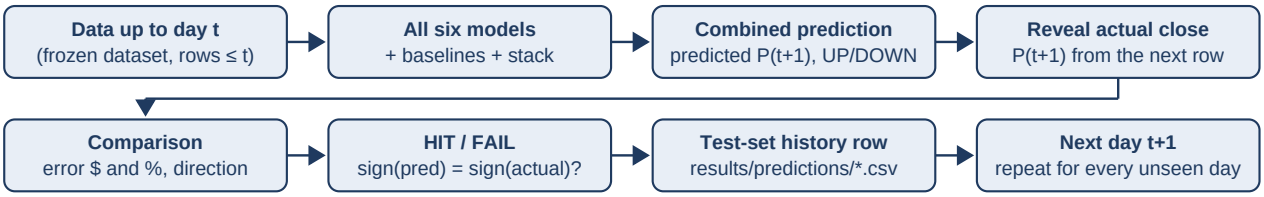


Figure 10. The verification loop implemented for the unseen test period (src/evaluation/backtesting.py :: main; interactively: prediction.py :: predict_for_date).

If the automatic record were restored (future work, §31): the removed prototype's design was a daily job calling the same `predict_next_day()` after each market's complete-bar cut-off, an append-only file keyed by (asset, as-of date) to prevent duplicates, and a verification step on the next run filling in the actual close, error and HIT/FAIL. Nothing in the current architecture prevents this; it was a scope decision.

20. HIT / FAIL Methodology

The project tracks **two separate things** for every verified prediction and never mixes them: the **direction outcome** (HIT/FAIL) and the **price error**.

```
actual_return r = ln(P_{t+1} / P_t)    predicted return ŷ (combined)
HIT  if sign(ŷ) == sign(r) and r ≠ 0     FAIL  if sign(ŷ) ≠ sign(r) and r ≠ 0     undefined ("flat")
if r == 0
error_usd = P´_{t+1} − P_{t+1}      error_pct = (P´_{t+1} − P_{t+1}) / P_{t+1} × 100    (reported as a number, never
thresholded)
```

- **No tolerance band.** A HIT requires only the correct sign; a prediction of +0.02 % on a day that closes +3 % is a HIT, and one of +0.5 % on a day that closes −0.1 % is a FAIL. No "within X %" price-tolerance rule is used anywhere, because such a rule is trivially satisfied by the random walk on calm days and would make the system look artificially accurate.
- **Zero-return days are excluded** from directional accuracy (their direction is undefined) — there are none in the stored test periods (n = 207 / 143 / 143 = all test days). In the per-day history file a zero actual return is stored as empty (excluded, exactly as in the statistics); in the dashboard's Predict-a-Day view it is shown as "flat".
- **Statistical reading.** The hit rate is reported with a one-sided binomial p-value against 50 %; with 143 days about 58 % is needed for p < 0.05, so the observed 47–54 % is chance-level and the report says so.
- **Price error is separate.** MAE, RMSE and MAPE in USD, plus the per-day error, are always shown next to the random walk's (whose error is the day's actual move), so that a "1.3 % average error" is not mistaken for skill.
- **Why this rule.** Sign agreement is the standard, tolerance-free definition of directional accuracy in the forecasting literature and is the only definition under which the random walk (no direction) is a fair reference.

21. Prediction History (per-day test predictions)

The prediction history is the file `results/predictions/<prefix>_test_predictions.csv` — one row per unseen test day — written by `src/evaluation/backtesting.py` and read by `prediction.py :: prediction_history()` for the dashboard. It is a **plain CSV file**; no database is used.

Field (CSV column)	Meaning
date	Day t — the last day whose data the prediction used ("Predicted on")
target_date	Day t+1 — the day whose close is predicted ("For")
prev_close	Actual close on day t (P_t), the anchor of the price reconstruction
actual_close	Actual close on the target date (P_{t+1})
actual_return	ln(actual_close / prev_close)
pred_return_<model>	Predicted log return, for each of: Naive, Naive-Mean, ARIMA, Ridge, RandomForest, LightGBM, CatBoost, GRU, LSTM, Stacked, Combined
pred_close_<model>	prev_close · exp(pred_return) — the predicted price
error_pct_<model>	(pred_close − actual_close) / actual_close × 100

Field (CSV column)	Meaning
direction_hit_<model>	1 if sign(pred_return) = sign(actual_return) else 0 (HIT / FAIL); empty on a flat day

Table 20. Fields stored for every unseen day (61 columns in the Gold file: the common columns, 4 per forecast, and the band columns of the Combined forecast).

Predicted on	For	Prev close	Predicted (Combined)	Actual	Error $	Error %	Pred. dir.	Actual dir.	Result	RW error %
2026-02-17	2026-02-18	4,882.90	4,890.62	4,986.50	-95.88	-1.92	UP	UP	HIT	-2.08
2026-02-18	2026-02-19	4,986.50	4,997.85	4,975.90	+21.95	+0.44	UP	DOWN	FAIL	+0.21
2026-02-19	2026-02-20	4,975.90	4,985.19	5,059.30	-74.11	-1.46	UP	UP	HIT	-1.65
2026-02-20	2026-02-23	5,059.30	5,067.16	5,204.70	-137.54	-2.64	UP	UP	HIT	-2.79
2026-09-04	2026-09-08	4,429.80	4,436.39	4,393.90	+42.49	+0.97	UP	DOWN	FAIL	+0.82
2026-09-08	2026-09-09	4,393.90	4,400.40	4,416.00	-15.60	-0.35	UP	UP	HIT	-0.50
2026-09-09	2026-09-10	4,416.00	4,421.47	4,364.50	+56.97	+1.31	UP	DOWN	FAIL	+1.18
2026-09-10	2026-09-11	4,364.50	4,370.09	4,366.20	+3.89	+0.09	UP	UP	HIT	-0.04

Table 21. First and last four rows of the Gold history (143 rows in total, 2026-02-18 → 2026-09-11), Combined forecast.

Asset	Rows (unseen days)	Period covered	Mean |error| % Combined / RW	Median |error| % Combined / RW	Largest miss % Combined / RW	HITs	Hit rate
Bitcoin	207	2026-02-18 → 2026-09-12	1.52 / 1.52	1.16 / 1.12	6.94 / 6.92	97 of 207	46.9 %
Gold	143	2026-02-18 → 2026-09-11	1.29 / 1.29	1.04 / 1.03	6.43 / 6.29	72 of 143	50.3 %
Silver	143	2026-02-18 → 2026-09-11	2.47 / 2.49	1.89 / 2.05	10.10 / 10.05	77 of 143	53.8 %

Table 22. Summary of the three history files (results/predictions/.csv).*

How records are created and kept. The file is produced in one pass by the evaluation script from the frozen dataset and the deployed models; it is **not** appended to day by day. Records are therefore immutable between evaluation runs — a row can only change if the whole evaluation is deliberately re-run (`make evaluate`), which regenerates the file from scratch and is recorded in the results tables. Duplicate rows cannot occur because each row corresponds to exactly one window end date. The dashboard uses these files for the Predict-a-Day chart and the predicted-vs-actual views of the Model Performance tab; the files also hold every single model's columns.

22. Real-World Forward Testing — what the project can and cannot claim

The principle. The strongest evidence for a forecasting system is a prediction that was fixed *before* the outcome existed and was compared with the outcome afterwards. Two levels of this exist in the project:

- **Frozen unseen test period (implemented).** The models were trained and tuned on data up to 2026-02-17 only. Every day from 2026-02-18 to 2026-09-11/12 is then predicted from data up to that day and compared with the actual next close. No model was trained or tuned on these days (enforced by the frozen split dates and the single evaluation script). The window was, however, inspected during development — when the close-time leak was found and when the Combined forecast and the band were introduced — so it is not a pristine one-shot holdout.
- **Fresh hold-out (implemented, 13 November 2026).** `src/experiments/fresh_holdout.py` runs the unchanged models on every close published *after* the frozen dataset — days that no version of the project had seen — with a decision rule fixed in advance (skill only if the Combined RMSE is below the random walk's and the Diebold–Mariano $p < 0.05$). Table 22b gives the result.
- **Live forward record (prototyped, removed).** A calendar-time log in which each forecast is written down before the market closes and verified the next day. This is stronger still, because it also tests the data pipeline in real operating conditions (vendor delays, contract rolls, holidays). The prototype was exercised only briefly before it was removed; the final system therefore **does not claim any live forward-testing record**.

What a short forward-testing period would and would not show. Even a fully implemented live record of a few weeks is evidence that the system *works operationally* — that predictions are produced on time, from complete bars, and verified correctly. It is **not** proof of long-term market profitability: with daily directional hit rates near 50 %, several hundred days are needed before a difference from chance becomes statistically visible, and market regimes change faster than that. The project's own 143–207-day frozen test period already illustrates this: the combined forecast is statistically indistinguishable from the random walk.

Asset	Days (target dates)	Forecast	RMSE vs random walk	DM p	Direction % (p)	UP calls % / up days %	MAPE %
Bitcoin	61 (2026-09-13 → 2026-11-12)	Combined (served)	-0.51 %	0.41	47.5 (0.70)	48 / 48	1.35
Bitcoin	61 (2026-09-13 → 2026-11-12)	Historical mean	+0.26 %	0.53	47.5 (0.70)	100 / 48	1.35
Bitcoin	61 (2026-09-13 → 2026-11-12)	Naive (random walk)	+0.00 %	—	—	0 / 48	1.34
Gold	44 (2026-09-14 → 2026-11-12)	Combined (served)	+1.34 %	0.29	43.2 (0.85)	100 / 43	1.06
Gold	44 (2026-09-14 → 2026-11-12)	Historical mean	+1.09 %	0.12	43.2 (0.85)	100 / 43	1.06
Gold	44 (2026-09-14 → 2026-11-12)	Naive (random walk)	+0.00 %	—	—	0 / 43	1.05
Silver	44 (2026-09-14 → 2026-11-12)	Combined (served)	+0.99 %	0.17	43.2 (0.85)	100 / 43	1.91
Silver	44 (2026-09-14 → 2026-11-12)	Historical mean	+0.78 %	0.09	43.2 (0.85)	100 / 43	1.90
Silver	44 (2026-09-14 → 2026-11-12)	Naive (random walk)	+0.00 %	—	—	0 / 43	1.89

Table 22b. results/experiments/fresh_holdout.csv — the frozen models on the days published after the frozen dataset. The verdict is the same as on the test window: no skill. About 40–60 days give the tests little power, and the live series is a fresh Yahoo download (a futures contract roll can add one artificial return).

23. Streamlit Application

`app/streamlit_app.py` (≈ 820 lines, single page with tabs). Every number it shows is read from the project's artefacts (`data/processed`, `data/models`, `results/`) or produced live by `src/inference/prediction.py`; nothing is hard-coded. Dark theme, Plotly charts, Streamlit metrics/dataframes; deep links `?asset=Gold&run=1` for demos.

Component	Purpose	What is displayed / user interaction
Sidebar	Control panel	Asset selector (Bitcoin / Gold / Silver); "Run prediction" (primary button); caption "runs all 6 trained models … no model to choose"; chart overlays (Bollinger, RSI, MACD; Volume for Bitcoin); "Sync live market data"; status caption (served forecast, best single model by validation, horizon, data source and last complete bar)
Overview strip	Context before any number	What is predicted (ticker, log return, number of features), the served forecast and its members, reliability on the unseen test days (direction %, MAPE vs random walk, DM p) — reads model_status.json
Tab 1 — Forecast	Live next-day forecast	Latest market data (last complete close with day change, 30-day high/low, 30-day σ); after "Run prediction": final card (predicted close, UP/DOWN, change % and $), calibrated 68 % band, live scorecard (each model's and the Combined prediction for tomorrow with its direction hits and average error vs the random walk on the last 20–90 days, re-computed on every press, plus a day-by-day expander), member agreement count, 60-day chart with forecast point and band, reliability table (Combined vs random walk), a warning if the vendor has not published the newest bar or if the download failed; full price history with train/val/test shading and optional indicator overlays and range buttons (6M/1Y/2Y/All)
Tab 2 — Predict a Day (unseen test)	Interactive out-of-sample demo	Select any day after the training data (stored test days plus every newer live day, marked NEW) → "□ Run models live" → five metrics (close on day t, predicted close for t+1, actual close, prediction error $ and %, direction HIT □ / FAIL □), context caption, actual-vs-predicted chart of the whole test period with the chosen day highlighted (diamond = prediction, circle = actual), table of each model on that day with its own error and HIT/FAIL
Tab 3 — Model Performance	Evaluation evidence	Test table of all 13 rows with the served and CV-selected rows labelled; "honest reading" box; bar charts (RMSE of return, direction %) vs the random-walk line; walk-forward table; predicted-vs-actual return line and scatter; feature-importance selector for Ridge/RF/LightGBM/CatBoost; all-asset table of the served forecast against every baseline; pooled walk-forward evidence; uncertainty-band calibration; expanders for regime table, design experiments E1–E6 and training diagnostics (over-fitting gap, loss curves)
Tab 4 — Methodology	Explanation for examiners	Target, data, split, features (with the close-time rule for the selected asset), models, how the combination works, metrics; details table of the served forecast (members, rule, walk-forward RMSE vs random walk, best single model, its stopping point and hyper-parameters)

Component	Purpose	What is displayed / user interaction
Footer	Positioning	The research-prototype / not-financial-advice disclaimer on every view

Table 23. Dashboard components (final code).

UI architecture. One Streamlit script; cached loaders (`st.cache_data`, 5 min) for the feature frames and result CSVs; session state for the last run result so that switching tabs does not re-run the models; all model logic lives in `src/` and is shared with the API and the tests. The dashboard never trains or evaluates — it only reads artefacts and calls the inference module.

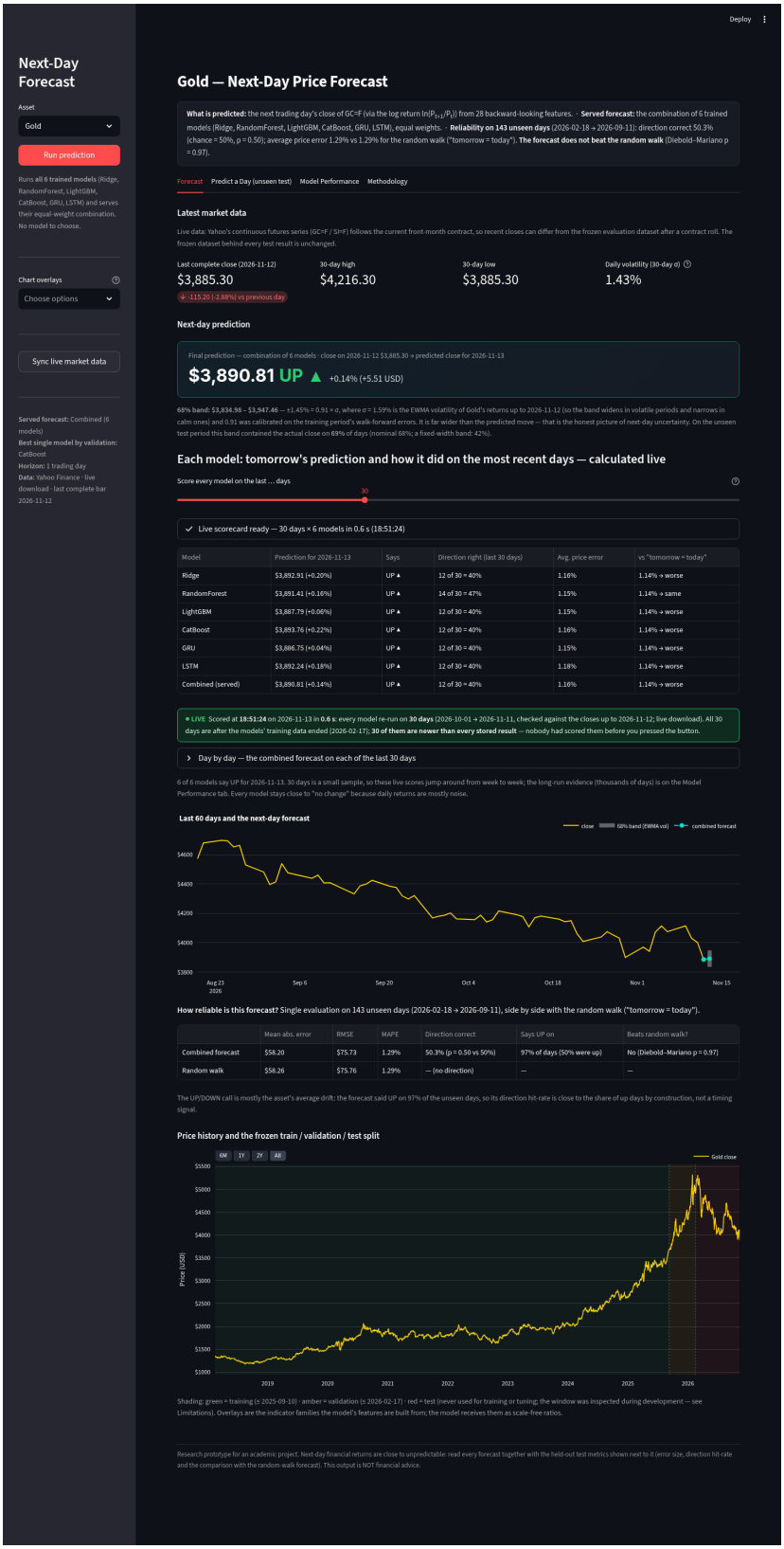


Figure 11. Forecast tab after "Run prediction" (Gold, captured from the final code on the report date): final combined prediction with the calibrated 68 % band, the live scorecard of every member, the 60-day chart, reliability vs the random walk, and the shaded split.

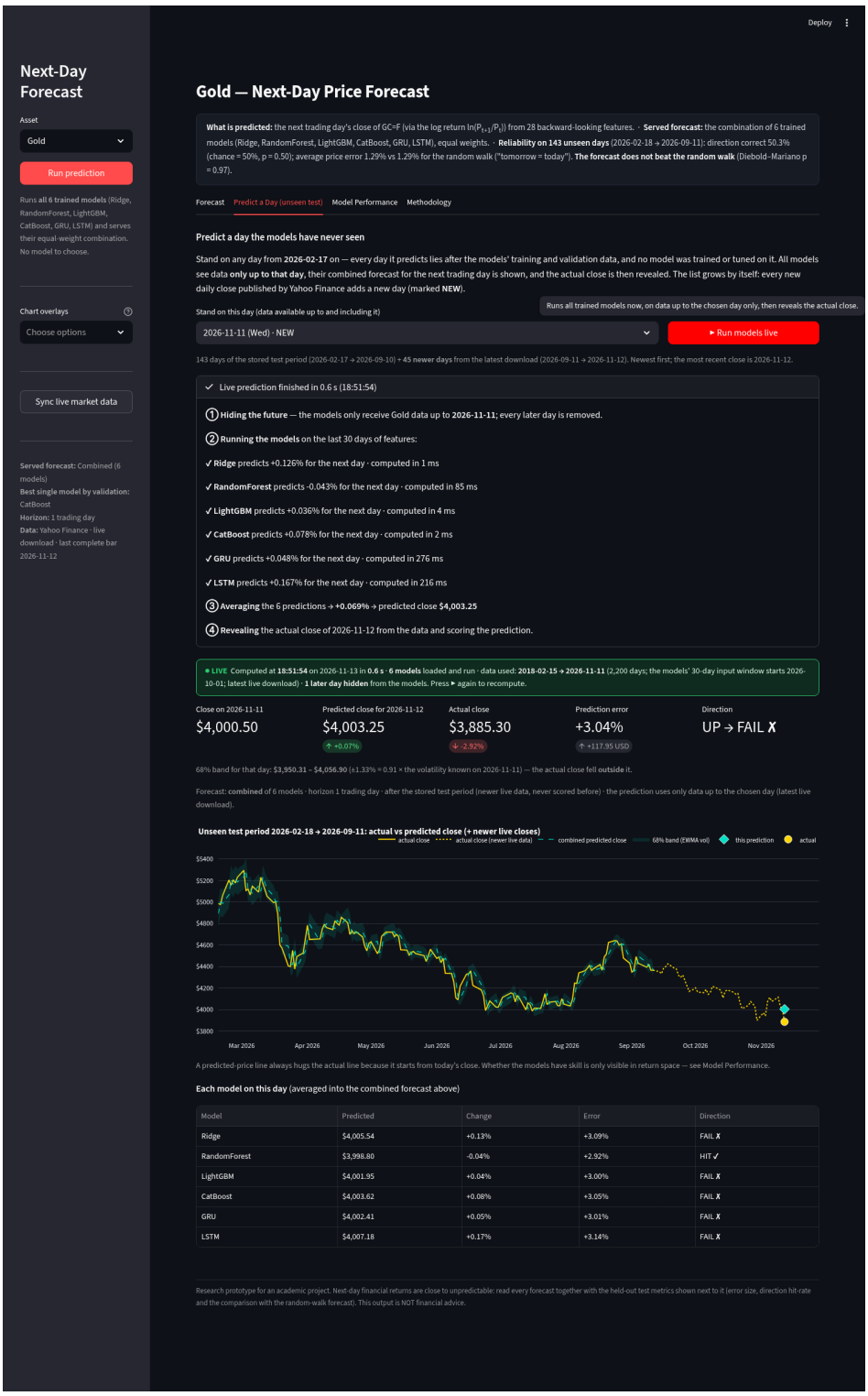


Figure 12. Predict a Day (unseen test): the combined forecast for the next trading day made from data up to the chosen day only, the revealed actual close, the error, HIT/FAIL and the band, and each member on that day.

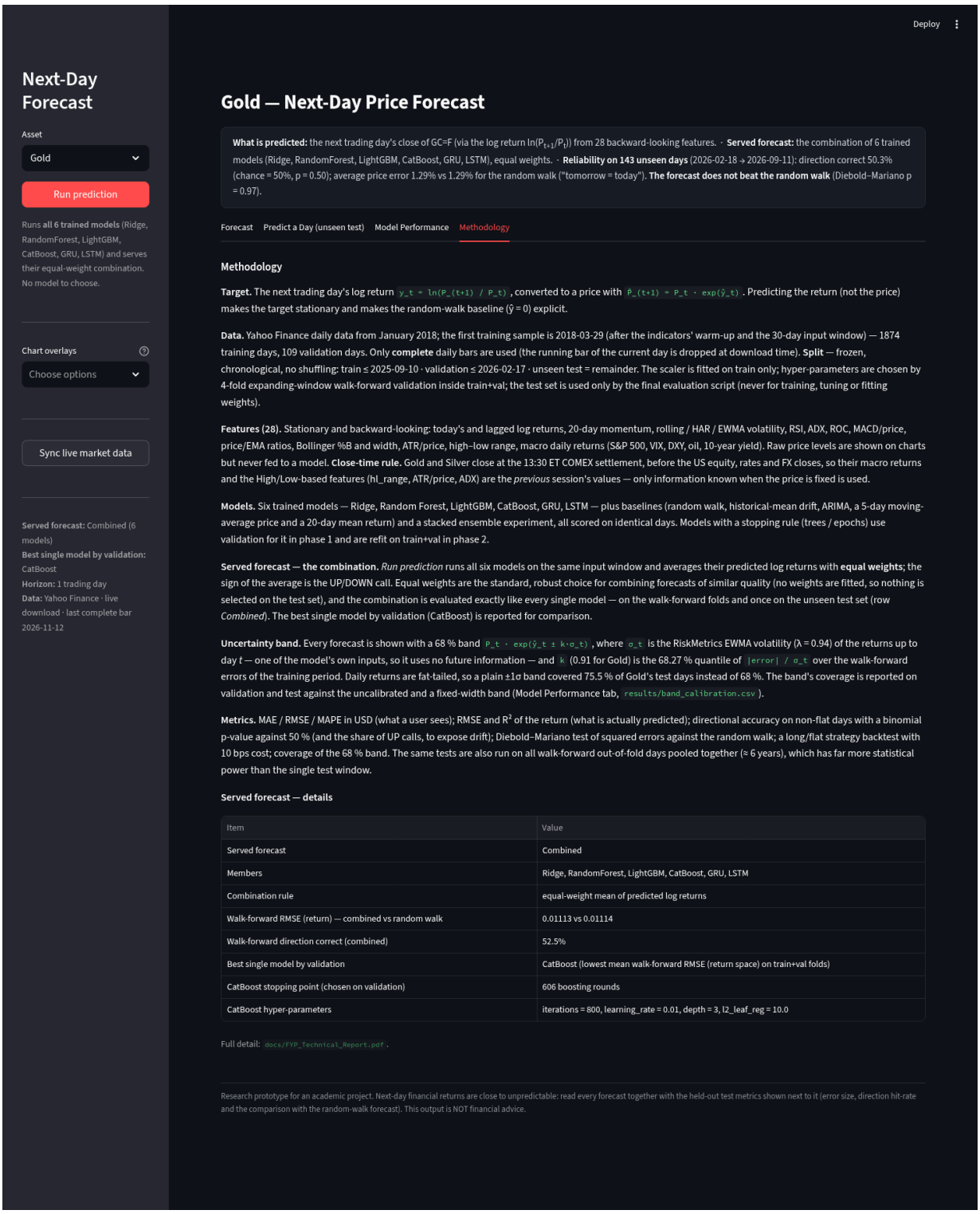


Figure 13. Methodology tab (Gold): target, data, split, features with the close-time rule, models, the combination rule and the band.



Figure 14. Model Performance tab (Gold): test table, honest reading, comparison charts, all-asset baseline table, walk-forward table, pooled evidence, return-space diagnostics, band calibration.

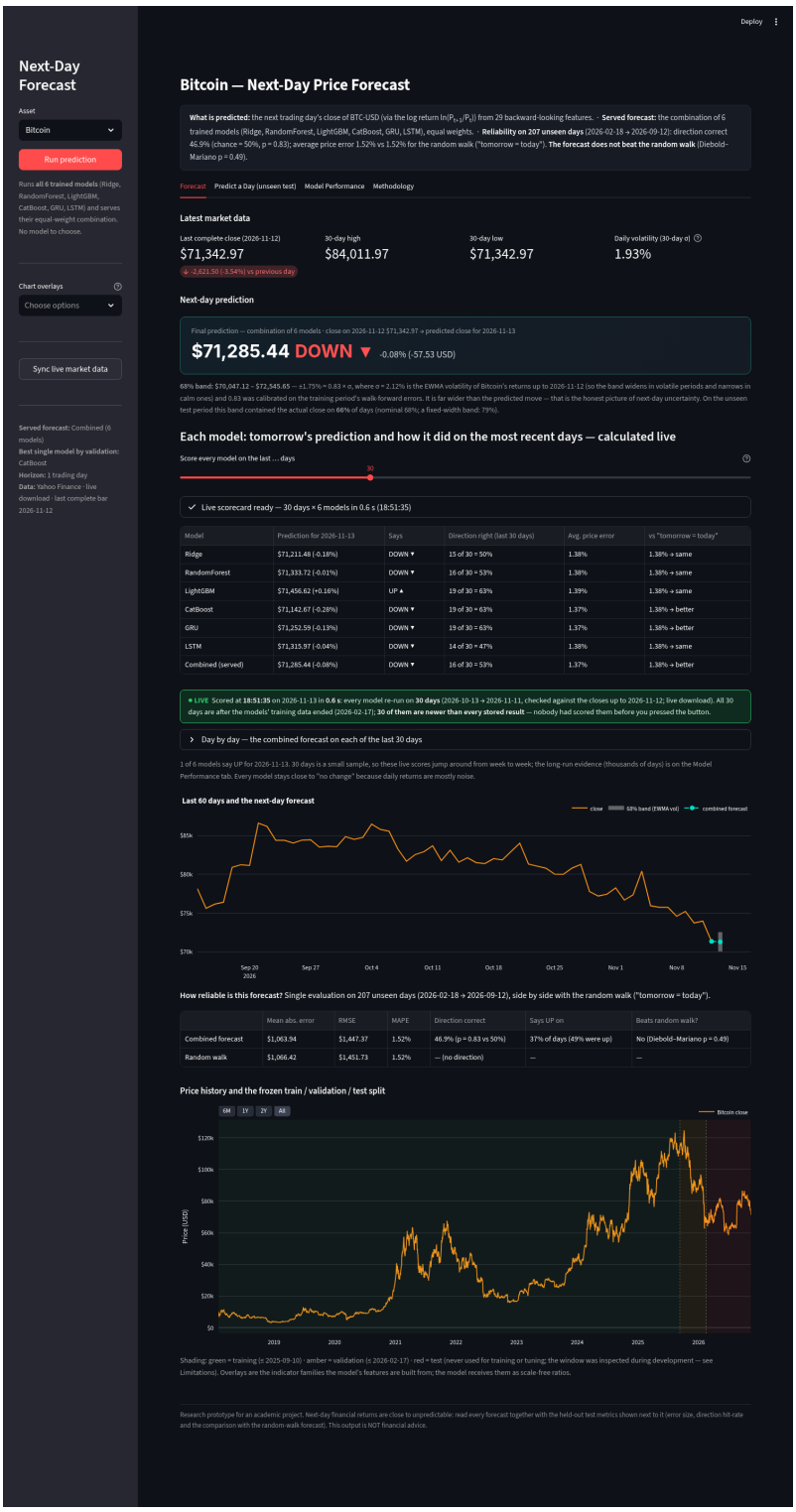


Figure 15. Forecast tab for Bitcoin (the Volume overlay is offered for Bitcoin only).

24. Visualisations

Chart (where)	Type	X-axis	Y-axis	Data represented	Why it is useful
Last 60 days + next-day forecast (Forecast tab)	Plotly line + markers	date	price (USD)	recent closes, dashed segment to the predicted close, 68 % band bar	Puts the forecast and its uncertainty on the price scale
Price history with split shading (Forecast tab)	Plotly line, subplots for overlays	date (range buttons)	price; RSI; MACD; volume	full stored history, train/val/test shading, optional Bollinger/RSI/MACD/volume	Shows what the models were trained on vs. the unseen period; indicator families behind the features
Actual vs predicted close, test period (Predict a Day)	Plotly lines + highlighted markers	target date	price (USD)	actual close and combined predicted close for every unseen day; the chosen day highlighted	Shows the demo prediction in context; caption warns that a price line always "hugs" the actual
RMSE (return) and direction % per model (Model Performance)	Plotly bar charts (2 panels)	model	RMSE of return; direction %	test-set values for every model (the far-off moving-average rules are listed in the caption) with the random-walk / 50 % reference lines	Model comparison at a glance; makes the "no model beats the random walk" result visible
Daily returns: actual vs predicted; scatter (Model Performance)	Plotly line; scatter with 45° line	target date; actual return %	return %; predicted return %	combined forecast vs realised returns on the test period	Skill is only visible in return space; shows the prediction barely moves
Feature importance (Model Performance)	Plotly horizontal bars	normalised importance	feature	top-15 features of the selected tree/linear member	Which inputs a member relies on
Over-fitting gap; GRU/LSTM loss curves (expander)	matplotlib PNG	model / epoch	RMSE / loss	train vs validation RMSE; training vs validation loss per epoch	Training diagnostics
Report figures (results/figures/, generated by src/evaluation/plots.py)	matplotlib PNG	—	—	price history; CV and test model comparison; per-asset actual-vs-predicted, residuals, strategy vs buy-and-hold, feature importance, loss curves; EDA ACF/PACF, return distributions, volatility clustering	Used in the written reports; reproducible with make figures

Table 24. Every chart implemented in the dashboard and the report-figure script.

Charts that are **not** implemented: a live prediction-history error chart over calendar time (no persistent live history exists), a band-calibration plot (coverage is reported as a table), and any interactive chart of the archived experiments beyond their tables.

25. Technology Stack

Technology	Version (installed)	Purpose in this project
Python	3.11.15 (build machine); 3.11 in CI; 3.10–3.12 supported	All code (TensorFlow 2.16.2 has no wheels for Python 3.13+)
pandas / NumPy	2.3.3 / 1.26.4	Data cleaning, feature engineering, windows, result tables
scikit-learn	1.6.1	Ridge, RandomForestRegressor, MinMaxScaler, TimeSeriesSplit
LightGBM	4.6.0	Gradient-boosted trees (member)
CatBoost	1.2.10	Gradient-boosted trees (member)
TensorFlow / Keras	2.16.2 / 3.10.0	GRU and LSTM models
statsmodels	0.14.6	ARIMA baseline; ADF, ACF/PACF in EDA
SciPy	1.13.1	Binomial test, t-distribution (Diebold–Mariano), skew/kurtosis
ta	0.11.0	Bollinger Bands, ATR, ADX, ROC indicators
yfinance	1.2.0	Yahoo Finance downloads (OHLCV and macro series)
urllib (stdlib)	—	alternative.me Fear & Greed API (JSON, no key)
joblib	1.5.3	Persistence of sklearn/LightGBM models, scalers and the stacked meta-model
Streamlit	1.50.0	Dashboard
Plotly	6.7.0	Interactive dashboard charts
matplotlib	3.9.4	Report figures (results/figures/)
FastAPI / uvicorn	0.162.5 / 0.54.0	REST API (/health, /models, /predict/{asset})
pytest	9.2.0	55 automated tests
reportlab	5.1.0	PDF report generation (docs/build_technical_report.py, this report)
Make	—	Pipeline targets (preprocess, tune, train, stack, evaluate, figures, experiments, fresh-holdout, test, serve, api)
GitHub Actions	ci.yml	Import checks, preprocessing of the frozen raw data and the full test suite on every push / pull request (pinned lock file)
Storage	CSV, JSON, joblib .pkl, CatBoost .cbm, Keras .keras	No database: all data, artefacts and results are files under data/ and results/

Table 25. Technologies actually used (versions read from the environment that built this report; requirements-lock.txt pins them). Not used: SQLite or any database, Docker, cloud services, paid data APIs, GPU.

26. Project Architecture

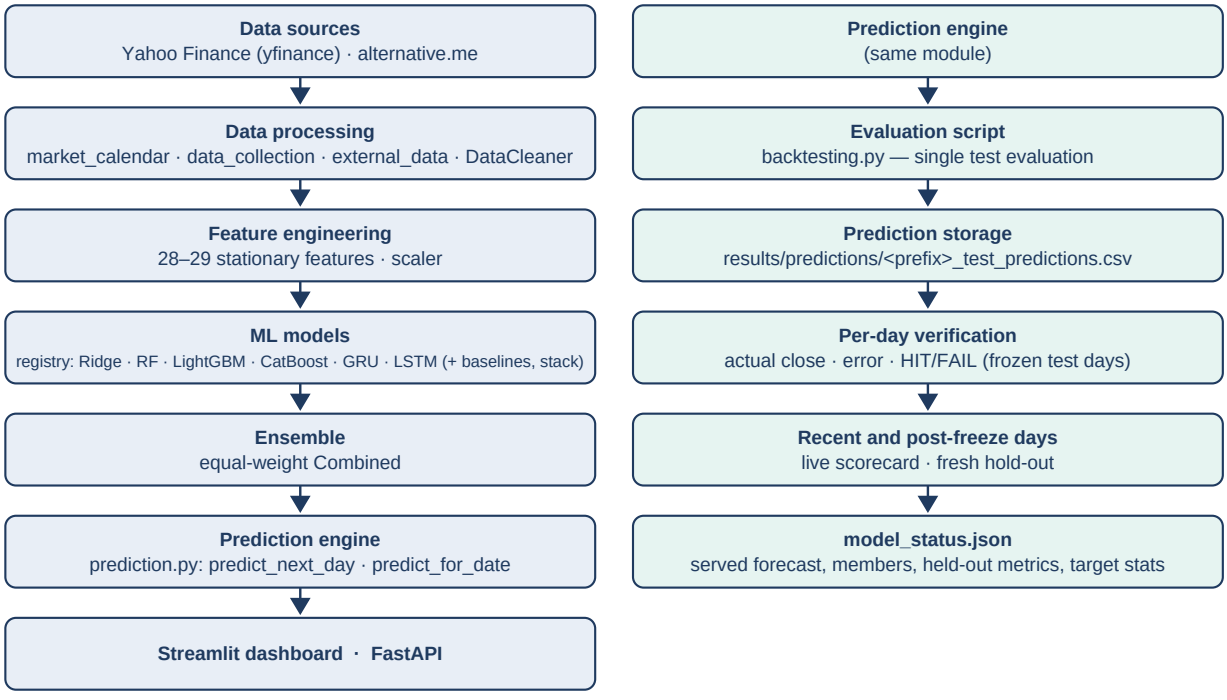


Figure 16. Left: the forecasting path. Right: the evaluation / verification path. Both share the prediction engine and the artefacts under data/ and results/.

Component	Responsibility	Inputs → outputs
Data source layer	Download complete daily bars from free public sources	tickers → data/raw/*.csv (frozen) or data/raw_live/*.csv (live)
Data processing	Clean, align calendars, apply the close-time rule, engineer features, split, scale	raw CSV → data/processed/*_features.csv, *_scaled.csv, data/models/*_scaler.pkl
Model layer (registry)	One interface for fit/predict/save/load of every model; two-phase fitting	dataset dict → data/models/* artefacts
Training & tuning	Walk-forward grid search; phase A / phase B training; diagnostics	best_params.json, train_val_metrics.csv, histories, feature_importance.csv
Evaluation	CV table, single test evaluation, regime analysis, model_status.json, FINAL_RESULTS.md, per-day predictions	results/*.csv, results/predictions/*.csv, data/models/model_status.json
Prediction engine	Load members, build the scaled window, combine, reconstruct price, attach held-out context; demo mode on frozen data	features (live or frozen) → result dict (price, return, direction, band, members, metrics, disclaimer)
Presentation	Streamlit dashboard; FastAPI JSON	result dicts + result files → UI / JSON
Quality	pytest suite, CI workflow, Makefile, pinned lock file	—

Table 26. Components of the architecture.

API. `GET /health`; `GET /models` (served forecast, members, CV-selected model and held-out metrics per asset); `GET /predict/{bitcoin|gold|silver}[?model=GRU]` (default = the Combined forecast with every member's prediction, target date, uncertainty band, data source, test metrics and disclaimer; 404 for unknown assets/models, 503 when artefacts are missing, 500 otherwise). The API shares `predict_next_day()` with the dashboard; it does not refresh live data itself (it uses whichever feature file is present).

27. File / Code Structure

File / folder	Handles	Purpose
config.py	configuration	Paths, tickers, DATA_START_DATE, target/tasks, frozen split dates, feature policy (LEVEL_COLUMNS, close-time rule), SEQ_LEN, default and tuned hyper-parameters, model_status loader
src/data/data_collection.py	data	Yahoo OHLCV download for the three assets (complete bars only)
src/data/external_data.py	data	Macro series (DXY, WTI, TNX, S&P 500, VIX) and Fear & Greed
src/data/market_calendar.py	data	Complete-bar cut-offs, next trading day, expected last complete bar
src/data/preprocessing.py	data · features · split	DataCleaner (cleaning + all features + split + scaler), create_sequences, build_dataset (the single loader)
src/data/sync_live_data.py	data (live)	Refresh into data/raw_live and write _live_features.csv atomically; never raises on API failure
src/data/eda.py	analysis	ADF tests, ACF/PACF, return distributions, volatility clustering → results/eda_summary.csv, figures
src/models/registry.py	models	Uniform model interface; Naive, Naive-Mean, ARIMA, Ridge, RandomForest, LightGBM, CatBoost, GRU, LSTM; save/load
src/models/model_gru.py, model_lstm.py	models	Keras architectures (single recurrent layer + dropout + dense)
src/models/ensemble_model.py	models (experiment)	Stacked ensemble: non-negative Ridge on out-of-fold predictions
src/training/tune_models.py	training	Walk-forward hyper-parameter search (train+val only) → results/tuning/
src/training/train_models.py	training	Two-phase training, validation metrics, loss curves, feature importance → data/models/
src/evaluation/cross_validation.py	evaluation	Expanding-window folds, cv_evaluate (with OOF predictions)
src/evaluation/backtesting.py	evaluation · storage	THE single test evaluation; Combined row; regime analysis; model_status.json; FINAL_RESULTS.md; per-day prediction files
src/evaluation/plots.py	visualisation	Report figures → results/figures/
src/experiments/run_experiments.py, audit_improvements.py, final_candidates.py	experiments	E1–E4, E5 and E6 (walk-forward / validation only, test set never read)
src/experiments/before_after.py, fresh_holdout.py	experiments	Before/after comparison on identical unseen days; the frozen models on the days after the frozen data (§22)
src/inference/prediction.py	prediction	predict_next_day (Combined), predict_for_date (demo), recent_track_record (live scorecard), prediction_history, live-data currency check
src/api/app.py	API	FastAPI service
src/utils/metrics.py	evaluation	RMSE/MAE/MAPE/R^2, directional accuracy + binomial p, Diebold–Mariano, strategy back-test, volatility metrics
src/utils/inverse_transform.py, reproducibility.py, logging_config.py	utilities	Price reconstruction; seeds; logging
app/streamlit_app.py	dashboard	The Streamlit application (4 tabs)
tests/ (6 files)	quality	55 tests: features/look-ahead, calendar, sequences/split, metrics, model contract, inference and failure modes
scripts/retrain.py	automation (pipeline)	End-to-end retraining: fetch → preprocess → tune → train → stack → evaluate → figures → experiments
Makefile · .github/workflows/ci.yml · requirements.txt · requirements-lock.txt	automation	Pipeline targets, CI, loose and exactly pinned dependencies
data/raw, raw_live, raw_3y_backup, processed, models	storage	Frozen raw data, live copies, earlier 3-year files, features/splits/scalers, 18 model artefacts + 3 scalers + 3 stacks + model_status.json
results/	storage	cv_results, final_test_results, train_val_metrics, feature_importance, regime_analysis, eda_summary, FINAL_RESULTS.md, tuning/, stacking/, predictions/, experiments/, histories/, figures/, archives
docs/	documentation	FYP_Technical_Report.pdf (this report) and its builder build_technical_report.py, the two study guides (PDF), figures/final (dashboard screenshots used here)
notebooks/ (4)	documentation	Companion notebooks that load and display the script outputs (exploration, features, training, evaluation)
catboost_info/	housekeeping	CatBoost training debris (git-ignored) — not used by the system

Table 27. Important files and folders.

28. Error Handling & Reliability

Situation	Behaviour of the final system	Assessment
Yahoo Finance / network failure during a live refresh	update_live_data() catches the exception, prints a message and returns False; the dashboard says that the download failed, names the stored dataset and its last bar, and continues with it (tested: test_live_sync_survives_network_failure; offline run verified 2026-11-13)	Handled
Raw CSV re-formatted by an editor (values padded with spaces)	Column names and values are stripped when read (preprocessing._read_raw_csv); a padded file gives byte-identical features (verified)	Handled (since 2026-11-13)
Empty response (rate limit)	fetch_asset() leaves the existing CSV untouched and returns an empty frame; treated as failure by the live sync	Handled
Vendor has not published the newest bar	live_data_is_current() compares the live file with the expected last complete bar; the dashboard warns and states which bar the forecast starts from; downloads are throttled to one per 30 min	Handled, explained to the user
Corrupt / partial download → invalid features	DataCleaner.validate_features() raises ValueError (NaN/inf/too few rows); the live file is not overwritten (atomic write via os.replace)	Handled
NaN in the newest feature row at prediction time	The scaler/model produce NaN → prediction.py raises ValueError("non-finite prediction") instead of showing a number (tested)	Handled
Missing processed data / models / evaluation	Explicit FileNotFoundError with the make target to run (preprocess / train / evaluate); dashboard shows an info/warning box; API returns 503	Handled
Feature-count mismatch between data and scaler	ValueError("Feature mismatch …") before any model runs	Handled
One member artefact broken	A missing artefact is simply not in the member list (the dashboard always states how many models were combined); an artefact that exists but fails to load or returns a non-finite value raises an explicit error instead of being skipped silently.	Handled
Unknown asset / model in the API	HTTP 404 with the list of valid names	Handled
Duplicate execution	Live refresh throttled; evaluation regenerates files idempotently; no scheduler exists, so duplicate scheduled runs cannot occur	n/a in the final system
Scheduler failure	No scheduler in the final system	n/a
Exchange holidays	next_trading_day() uses business days only — a forecast made on the day before a US holiday is labelled with the holiday date (the dashboard label says "next trading day")	Weakness (cosmetic)
Futures contract roll in the live series	Cannot be prevented; the dashboard states that live closes may differ from the frozen dataset after a roll	Weakness (documented)

Table 28. Failure modes and how the code responds.

29. Limitations

- **Markets are close to unpredictable at a one-day horizon.** This is the dominant fact: lag-1 return autocorrelation is -0.05 / -0.03 / -0.02; the project's answer to its own research question is negative.
- **External events are not modelled.** News, regulation, exchange outages and macro shocks are outside the feature set (only their reflection in past prices, macro returns and one sentiment index).
- **Historical patterns may not persist.** The 2026 test window is a different regime from 2018–2025 (Gold and Silver traded far above their training ranges; Silver's 30-day volatility exceeded every training value on most test days). Stationary features reduce but do not remove this.
- **One test window, limited sample, inspected during development.** 143–207 unseen days: with 143 days, 56 % direction is not distinguishable from 50 %; 36 model-rows are reported, so one p ≈ 0.02 cell is expected by chance. The window was inspected during development (leak fix, Combined forecast, band), so it is not a pristine one-shot holdout. The pooled walk-forward record (≈ 6 years, §14.5) and the fresh hold-out (§22) are the cleaner evidence — and in both no forecast is distinguishable from the random walk either.
- **No persistent live forward-testing record.** The automatic daily record was removed; the fresh hold-out (§22) covers only ≈ 40–60 days so far.
- **Phase-A validation metrics are mildly optimistic.** LightGBM, CatBoost, GRU and LSTM choose their stopping point on the validation set they are then scored on (Table 12, Figure 6). Model selection uses the walk-forward folds instead, and the test, pooled and fresh results are unaffected.

- **Walk-forward folds share the training-split scaler** (and target mean/std), so the early folds are scaled with statistics that include later training data. This affects only model selection, the pooled evidence and the band's k, and only the scale-sensitive Ridge/GRU/LSTM; any bias favours the models.
- **Data limitations.** Yahoo Finance is an unofficial free source; BTC-USD is a composite index; GC=F / SI=F are continuous front-month series with roll effects and unusable volume; Yahoo revises a few historical closes between downloads (mean < 0.005 %, max 0.75 % on one Bitcoin day) and publishes Bitcoin's previous-day bar with a delay; the metals' macro features are one day old by construction (close-time rule); Fear & Greed is a proprietary composite.
- **A simple uncertainty band.** The band $k \cdot \sigma_t$ is calibrated on training-period errors and close to nominal on the test period, but it is a single-parameter volatility rule, not a full predictive distribution, and it under-covers in sudden volatility jumps (Silver, validation window).
- **Asset-specific.** Bitcoin's 7-day calendar vs the metals' exchange calendar; exchange holidays are not modelled for the target-date label; Silver depends on gold's same-day return being available.
- **Modelling choices.** Small hyper-parameter grids; a single 32-unit recurrent layer; selection by mean fold RMSE (a practitioner might select on direction or Sharpe); the stacked ensemble is a linear blend; the combination is an equal-weight mean, which is a ceiling when the members carry no signal.
- **Evaluation caveats.** The strategy back-test is stylised (long/flat, flat 10 bps, no slippage, shorting, sizing or financing); the Diebold–Mariano test assumes covariance-stationary loss differentials, only approximate with fat-tailed returns; the before/after comparison reads the test rows a second time (declared, no selection made on it).

30. Ethical / Academic Positioning

This is an academic machine-learning prediction and evaluation project. It is **not** financial advice, not an investment strategy, not a guaranteed or profitable trading system and not a professional forecasting service. The disclaimer is embedded in the code (`prediction.DISCLAIMER`) and is returned with every API response and shown in the dashboard footer.

Responsible interpretation. Every forecast is displayed together with its held-out error, its direction hit-rate with a p-value and its comparison with the random walk, precisely so that a reader cannot mistake a plausible-looking number for skill. The project reports negative results, archives superseded results with an explanation instead of deleting them, and documents the leak it found in its own earlier version. Anyone using the system should treat the predicted price as "today's close plus noise of the size shown in the band" — which is what the evaluation says it is.

31. Future Improvements

- **Restore and run the automatic daily forward record** for a longer period (months, not days) — the prototype design exists (§19); it would turn the operational pipeline into a continuously growing out-of-sample history.
- **Longer and more diverse test evidence:** several rolling test windows or a full walk-forward re-training schedule over 2022–2026 instead of one frozen window, to show robustness across regimes.
- **Volatility forecasting as a product:** experiment E4 showed 22-day realised volatility *is* more predictable than the return (Bitcoin Ridge −11.5 % RMSE vs persistence, though only ~4 % better than EWMA and worse than EWMA for Silver); a served volatility band would be more useful than a point price.
- **Richer uncertainty:** conformal or quantile-regression intervals, or a GARCH-type volatility model, in place of the single-parameter $k \cdot \sigma_t$ band.
- **Additional information sources** with verified timestamps: order-flow / on-chain metrics for Bitcoin, positioning data (COT) for the metals, news sentiment — every new series must pass the close-time check.
- **More data:** intraday bars would give many more samples per year and permit realised-volatility features at finer resolution.
- **Modelling:** larger tuning budgets with nested CV, temporal-fusion or attention models only if the data volume justifies them, and probabilistic (distributional) losses.
- **Engineering:** exchange-holiday calendar for the target-date label, a small results database if a live history is kept, scheduled re-training, and hosting of the dashboard.

32. Final Project Status — honest assessment

Question	Answer	Evidence
Is the system fully functional?	Yes. Data download, preprocessing, tuning, training, evaluation, dashboard, API and tests all run from the Makefile; a live prediction was produced for all three assets on the report date.	make pipeline; make predict output (§17.1); 55/55 tests, run by CI on every push
Is the dataset pipeline correct?	Yes. Complete bars only, correct calendars, no nulls/duplicates, close-time alignment; the earlier synthetic-weekend and intraday-snapshot problems were found and fixed.	§6–7, §11; tests test_features.py, test_market_calendar.py
Are the features correctly implemented?	Yes. 28–29 backward-looking stationary features; look-ahead and close-time tests pass; ablation shows no harmful group.	§8; test_no_lookahead_in_any_feature and two close-time tests
Are the models correctly implemented?	Yes. Uniform registry, two-phase training with validation-only early stopping, saved/loaded artefacts reproduce the evaluation; model contract tested.	§12–13; test_models_and_inference.py
Is the evaluation methodology valid?	Yes. Frozen chronological split, walk-forward tuning/selection on train+val, single test evaluation, return-space metrics, DM and binomial tests, baselines.	§10, §14–15
Is there data leakage?	None known after the audit. The one leak found (same-day macro/High-Low for the metals) was fixed, tested and its effect documented. The before/after script re-reads the test rows without selecting anything (declared).	§11; Table 10
Is the ensemble working correctly?	Yes. The served forecast is the equal-weight mean of six members; the stored Combined predictions equal the mean of the stored member predictions (tested); it is evaluated like a single model.	§16; test_combined_test_row_is_the_mean_of_its_members
Is automatic prediction working?	Not present. It was built, verified and removed at the author's request; the final system produces predictions on demand only.	§19
Is prediction history working?	Yes for the frozen unseen test period (143–207 rows per asset with predicted/actual/error/HIT-FAIL), the live scorecard on the most recent days and the fresh hold-out. No persistent live log.	§21–22; results/predictions/*.csv; results/experiments/fresh_holdout.csv
Is the dashboard complete?	Yes: four tabs, one-click combined forecast with the calibrated band and a live scorecard, an unseen-day demo that grows with every new close, full performance and methodology views, disclaimer. Verified to run without errors for all three assets, online and offline (2026-11-13).	§23; Figures 11–15
Is the project suitable for FYP demonstration?	Yes. It demonstrates the full life-cycle, a defensible methodology, a working product and scientific honesty about a negative result.	whole report; docs/FYP_Phase2_Final_Study_Guide.pdf
Strongest parts	Leakage audit and the discovered close-time leak; walk-forward validation with formal tests; the equal-weight combination that removes model cherry-picking; reproducibility (frozen dates, seeds, single loader, tests, CI); a dashboard that shows reliability next to every number.	—
Weakest parts	No predictive skill over the random walk (expected, but it is the result); one frozen test window that was inspected during development (mitigated by the pooled and fresh evidence); no persistent live record; a simple single-parameter band; free-vendor data quirks (contract rolls, delayed bars, degenerate futures bars).	—
Audit of 13 November 2026 — what was fixed	(1) data/raw/bitcoin_data.csv (and three backup files) had been re-formatted with padded spaces by an editor, which stopped preprocessing with KeyError: 'timestamp' — restored byte-identical from git, and the loader now tolerates padding; (2) the CI workflow never built the processed data, so 10 tests failed on every push — fixed, CI now runs preprocessing and the full suite on a pinned environment; (3) the lock file lacked five packages and Python 3.13+ cannot install TensorFlow 2.16 — lock regenerated, Python 3.10–3.12 documented; (4) this report updated to the final system; (5) dashboard wording made exact (offline message, test-window caveat); (6) the fresh hold-out added. Every test-set result was re-verified unchanged.	§22, §28; README
Before the viva	Commit and push the audit changes (CI should turn green); rehearse the honest framing of the negative result; have the frozen dataset ready in case the venue has no internet (the dashboard then states that it uses the stored data).	—

Table 29. Final status.

33. Final Technical Summary

Problem → predict the next trading day's close of Bitcoin, Gold and Silver and determine whether machine learning beats the random walk at a one-day horizon. **Dataset** → Yahoo Finance daily OHLCV from 2018-01-01 (3,177 / 2,186 / 2,186 raw rows; complete bars only) plus DXY, WTI, 10-year yield, S&P 500, VIX and the crypto Fear & Greed index; no nulls, no duplicates, exchange calendar kept for the metals. **Features** → 29 / 28 / 29 stationary, backward-looking features (returns and lags, momentum, rolling/HAR/EWMA volatility, RSI/ADX/ROC, normalised MACD, EMA ratios, Bollinger %B/width, ATR/price, high–low range, macro returns, sentiment/calendar/volume for Bitcoin, gold return for Silver) under a verified close-time rule; price levels excluded. **Target** → next-day log return $\ln(P_{t+1}/P_t)$, converted to a price; UP/DOWN from its sign. **Models** → Ridge, Random Forest, LightGBM, CatBoost, GRU, LSTM (trained) against random-walk, historical-mean, ARIMA, MA-5 and Mean-20d baselines and a stacked-ensemble experiment. **Training** → frozen chronological split (train ≤ 2025-09-10, validation ≤ 2026-02-17, test after), MinMax scaler fitted on train, 4-fold expanding walk-forward hyper-parameter search on train+val, two-phase training (validation-monitored stopping, refit on train+val), seeds and pinned versions. **Evaluation** → one evaluation on 207 / 143 / 143 unseen days with USD and return-space errors, directional accuracy with a binomial test, Diebold–Mariano test against the random walk, strategy back-test and regime analysis. **Ensemble** → the served forecast is the equal-weight mean of the six models' predicted returns — no fitted weights, no model selection by the user. **Prediction** → one click (or one API call) refreshes the last complete bar, runs all six models and returns one price, UP/DOWN, a calibrated 68 % band and each member's prediction with the held-out reliability next to it. **Dashboard** → Streamlit, four tabs (Forecast with a live scorecard, Predict a Day on unseen days, Model Performance, Methodology) plus a FastAPI endpoint. **Automation** → Makefile pipeline, retrain script, 55 tests run by CI on a pinned environment; the automatic daily forward-prediction record was prototyped and removed from the final system. **Verification** → every unseen test day is predicted from data up to that day and compared with the actual next close (error $ / %, HIT/FAIL), stored as a per-day history and shown in the dashboard. **Results** → the combined forecast's next-day error is 1.52 % / 1.29 % / 2.47 % of price — the same as the random walk's — with directional accuracy 46.9 / 50.3 / 53.8 % and Diebold–Mariano p = 0.49 / 0.97 / 0.46: no model, single or combined, beats the random walk at a one-day horizon in this data — confirmed on ≈ 6 years of pooled walk-forward predictions and on the days published after the data were frozen. The project's value is a correct, tested, transparent and reproducible pipeline that reports that result honestly — including the discovery and correction of a close-time data leak that had produced an apparently significant 62 % result.

End of report. Regenerate with `python docs/build_technical_report.py`; *all tables are produced from the result files at build time.*